

게임서버 프로그래머 포트폴리오

작성자: 김형섭

email : polaszx3@gmail.com

blog : <https://trynbug.github.io/>

목차

1. MMO 게임 서버 C++
2. IOCP 네트워크 라이브러리 C++
3. 게임 콘텐츠 전용 IOCP 네트워크 라이브러리 C++
4. Tool : 서버 모니터링 클라이언트 C#
5. Lockfree Stack C++
6. TLS 메모리풀 C++
7. A* 알고리즘 길찾기 프로그램 C++
8. Jump Point Search 알고리즘 길찾기 프로그램 C++

프로젝트 소스코드 : https://github.com/TrynBug/HSKim_proj

1. MMO 게임 서버

개발환경

- Windows, C++, Redis, MySQL

개발기간

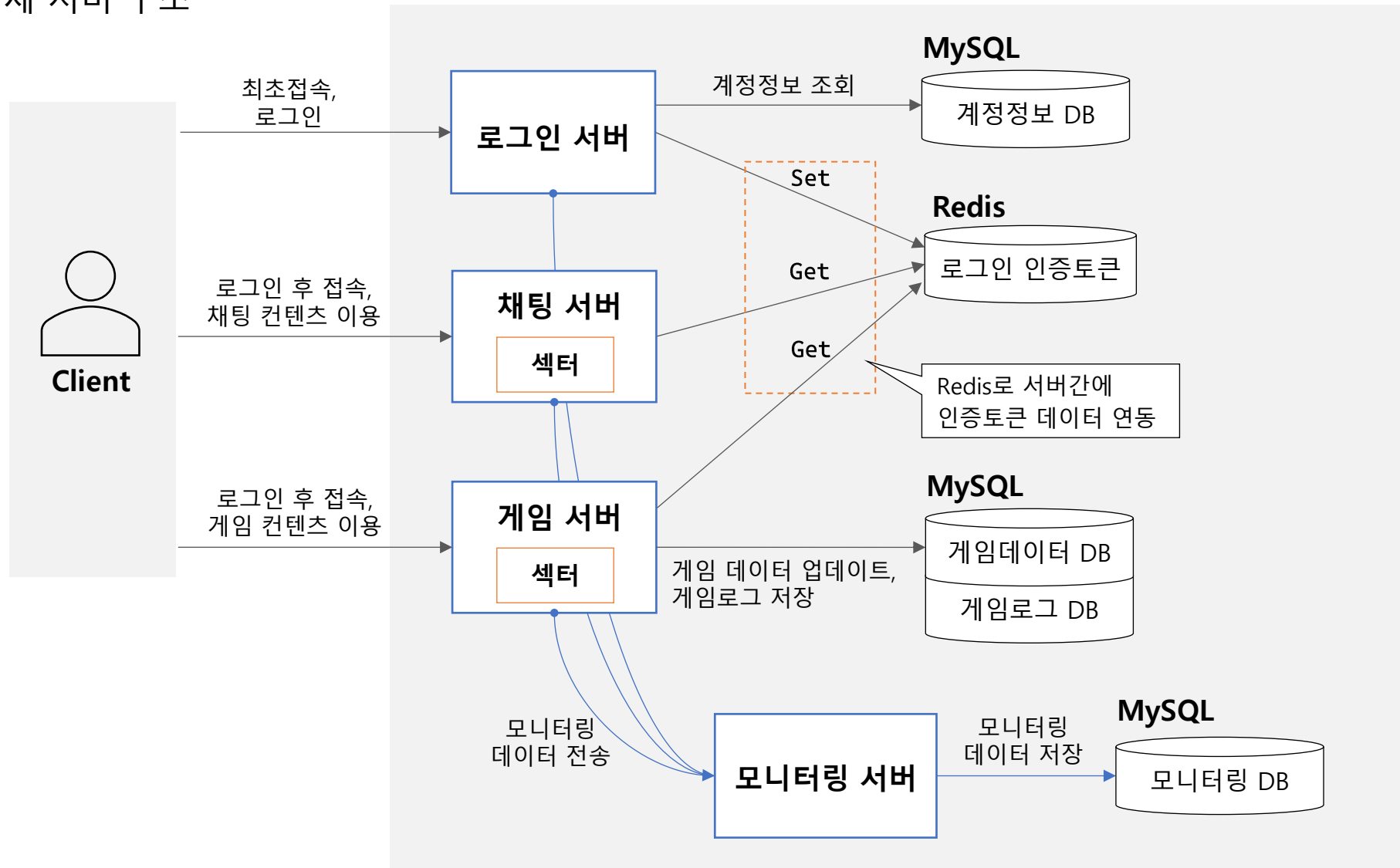
- 2023.01 ~ 2023.05

개요

- 자체 개발한 IOCP 네트워크 라이브러리 사용
- 게임 서버, 로그인 서버, 채팅 서버, 모니터링 서버를 각각 개발하여 서로 연동되어 작동하도록 함
- 서버간의 빠른 데이터 연동을 위해 Redis 인메모리 DB 사용
- 게임 데이터 업데이트, 게임 로그 저장 등을 위해 MySQL DB 사용
- 서버 오류 검증을 위해 5000명의 더미 플레이어를 넣어 5일간 테스트를 수행함
- 게임플레이 영상: <https://youtu.be/91Fad3ZqJuw> (※ 클라이언트는 직접 개발하지 않았습니다. 제공받은 것을 사용하였습니다.)

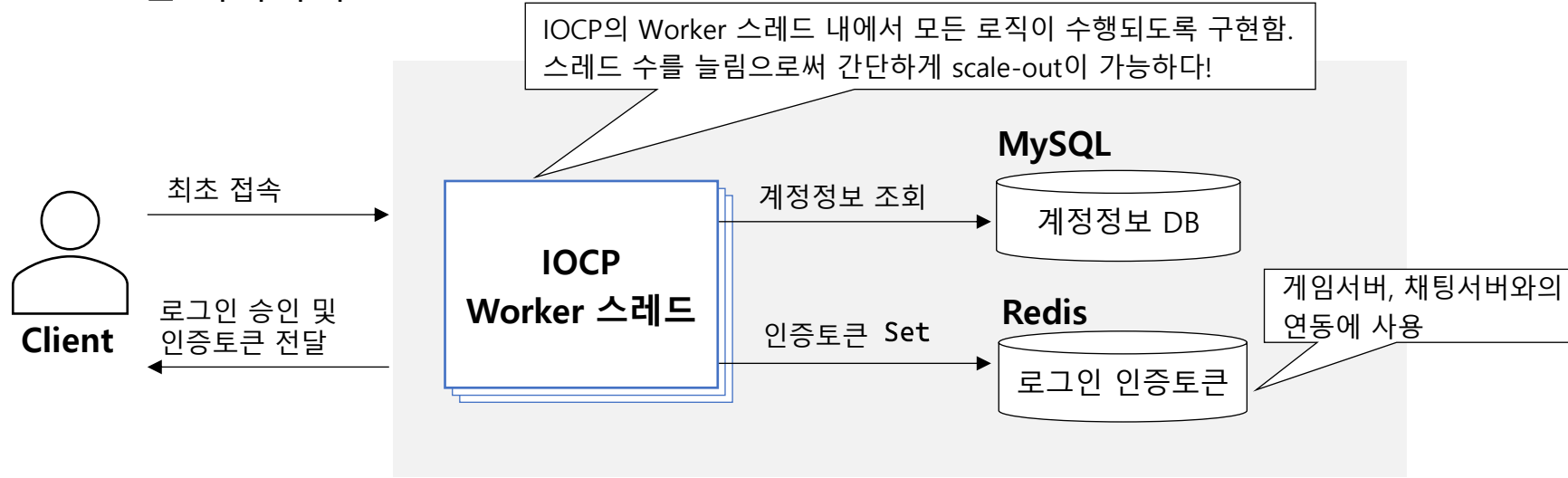
1. MMO 게임 서버

전체 서버 구조

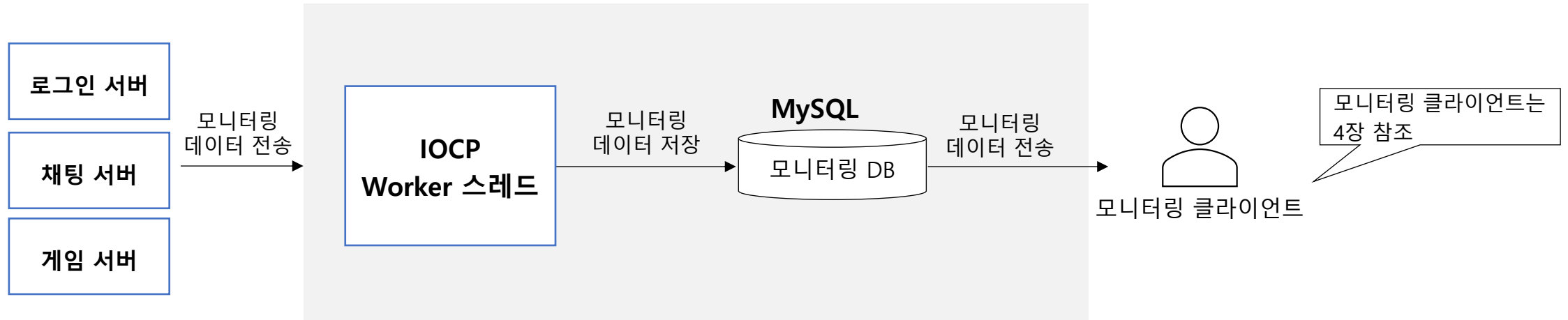


1. MMO 게임 서버

로그인 서버의 구조

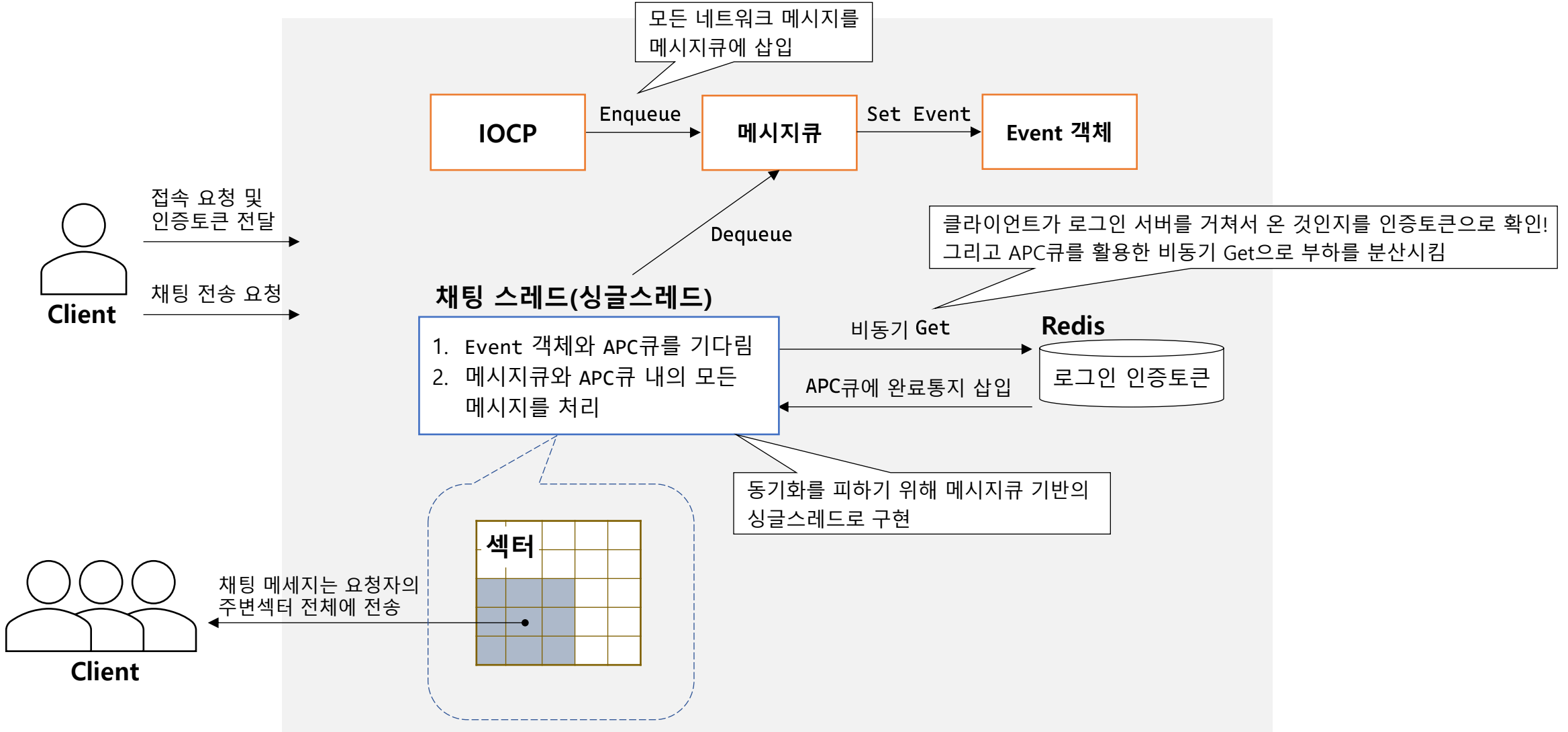


모니터링 서버의 구조



1. MMO 게임 서버

채팅 서버의 구조

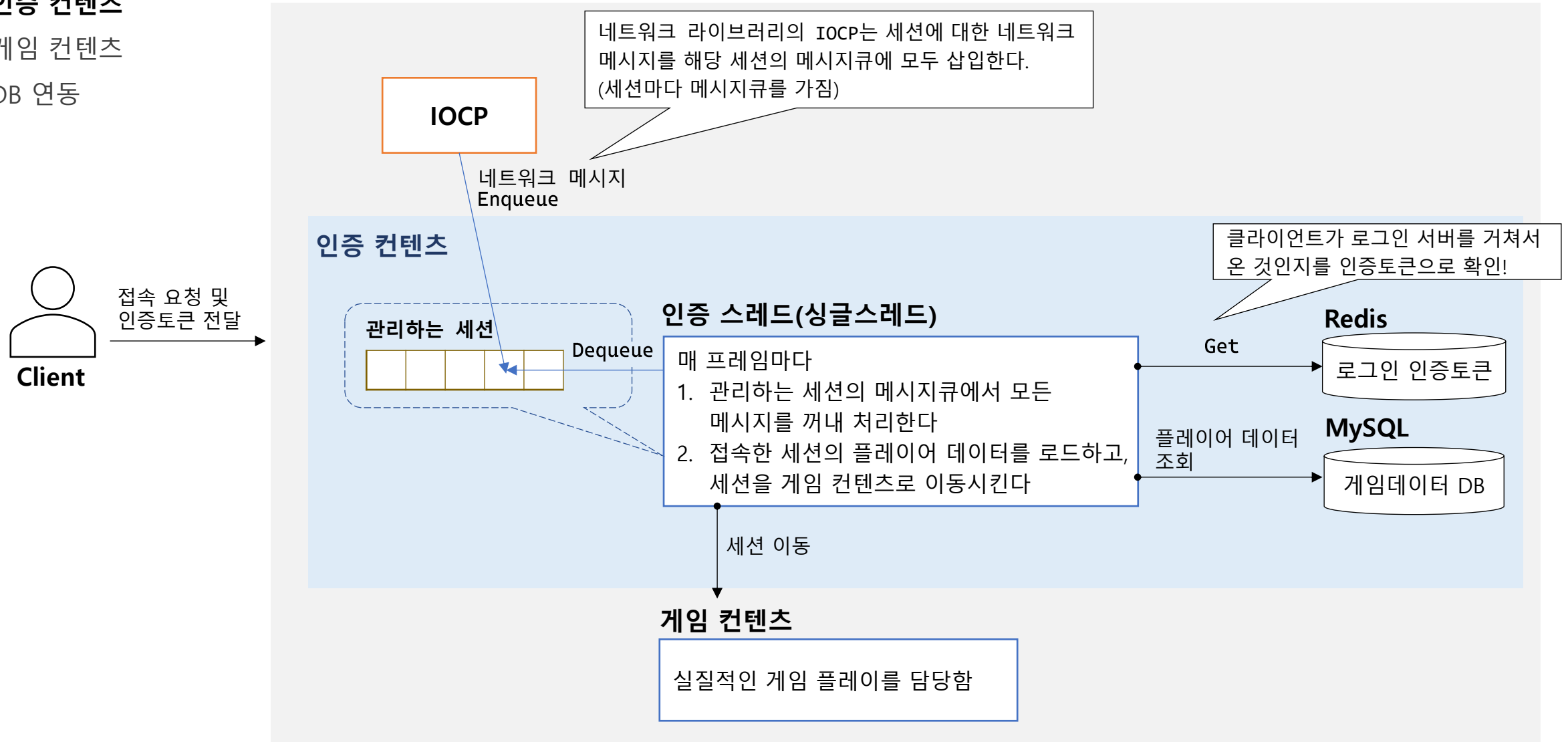


1. MMO 게임 서버

게임 서버의 구조

✓ 인증 콘텐츠

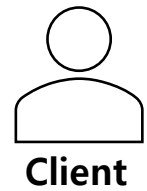
- 게임 콘텐츠
- DB 연동



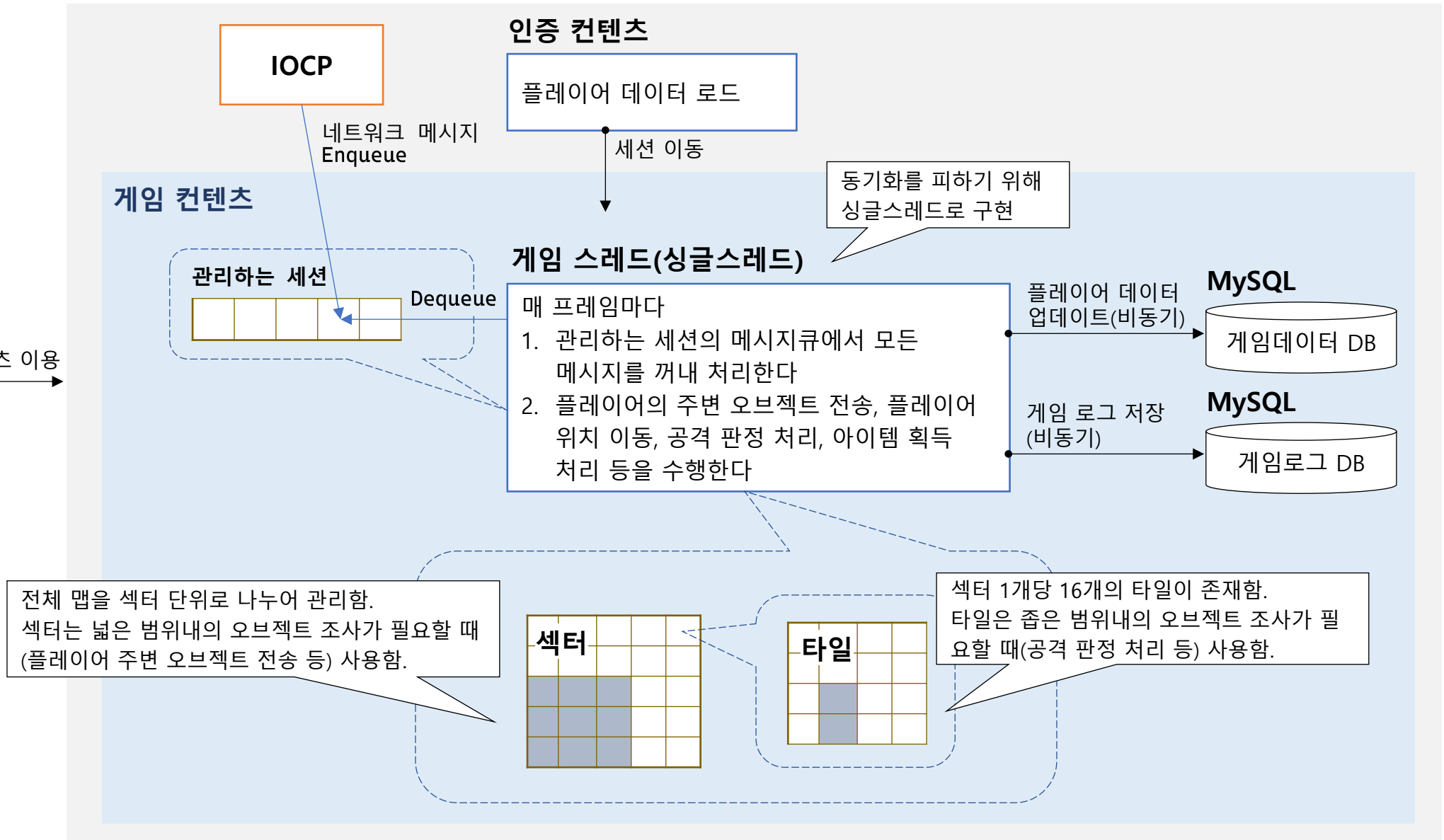
1. MMO 게임 서버

게임 서버의 구조

- 인증 콘텐츠
- ✓ 게임 콘텐츠
- DB 연동



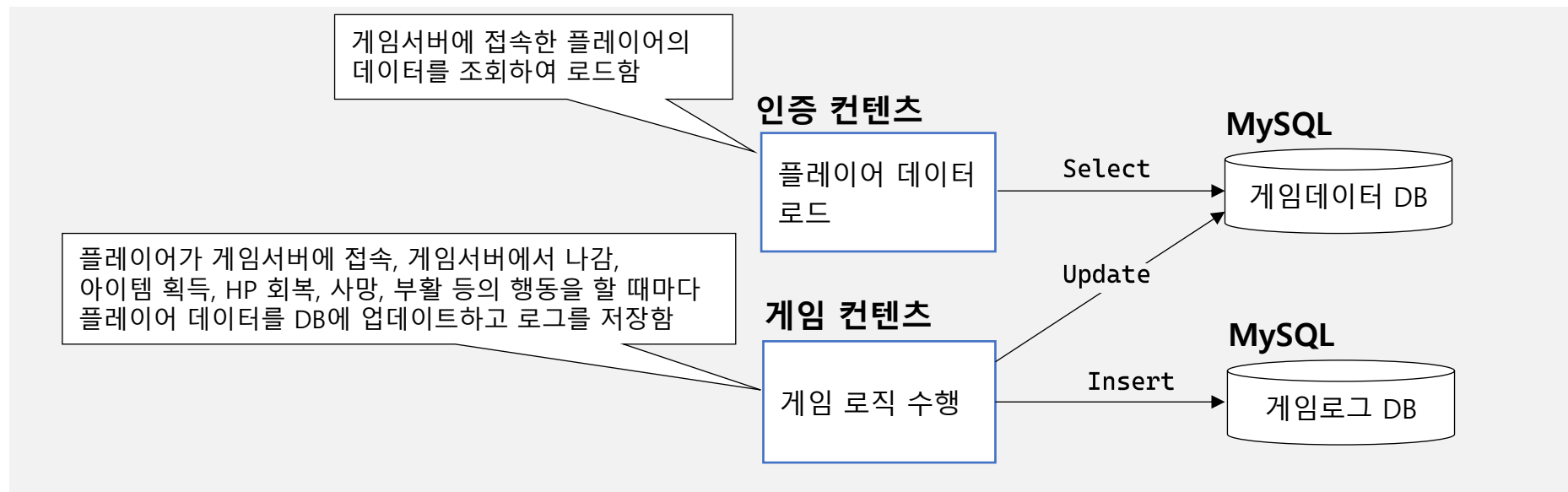
게임 콘텐츠 이용



1. MMO 게임 서버

게임 서버의 구조

- 인증 콘텐츠
- 게임 콘텐츠
- ✓ DB 연동



DB에 저장된 게임로그의 모습

로그 데이터의 의미를 'type' 과 'code' 컬럼으로 구분하고, 나머지 컬럼들에 값을 저장하였음.

```
10 • select * from `logdb`.`gameLog`;
```

no	logtime	accountno	servername	type	code	param1	param2	param3	param4	message
3160736	2023-02-16 17:58:35	14907	Game	3	31	103	111	0	NULL	NULL
3160737	2023-02-16 17:58:35	14966	Game	1	11	84	178	0	0	10.0.1.2:48630
3160738	2023-02-16 17:58:35	14966	Game	3	31	91	113	0	NULL	NULL
3160739	2023-02-16 17:58:35	11414	Game	1	11	226	176	0	0	10.0.1.2:48886
3160740	2023-02-16 17:58:35	11414	Game	3	31	96	93	0	NULL	NULL
3160741	2023-02-16 17:58:35	11109	Game	1	11	98	168	0	0	10.0.1.2:49142
3160742	2023-02-16 17:58:35	11109	Game	3	31	99	107	0	NULL	NULL
3160743	2023-02-16 17:58:35	11742	Game	1	11	382	136	0	0	10.0.1.2:49910
3160744	2023-02-16 17:58:35	11742	Game	3	31	104	105	0	NULL	NULL
3160745	2023-02-16 17:58:36	14217	Game	1	12	172	28	0	0	NULL
3160746	2023-02-16 17:58:36	14892	Game	3	31	160	16	0	NULL	NULL
3160747	2023-02-16 17:58:36	10216	Game	3	31	78	164	0	NULL	NULL

1. MMO 게임 서버

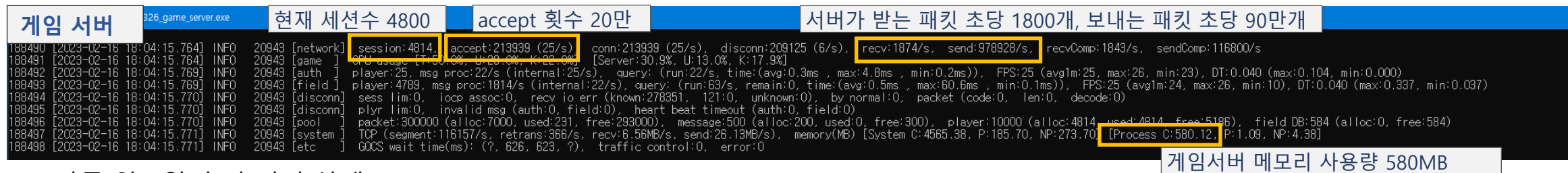
서버 오류 검증 테스트

테스트 조건

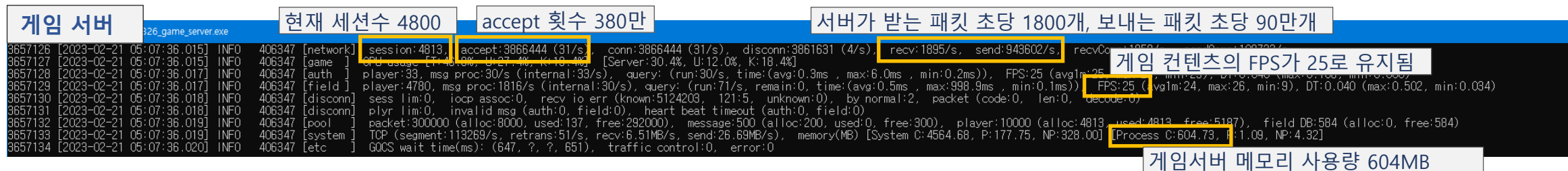
- 더미 클라이언트 프로그램을 사용하여 5000명의 클라이언트 연결
더미 클라이언트는 서버와 통신하면서 서버가 잘못된 패킷을 보내지 않았는지, 연결을 잘못 끊지 않았는지 등을 체크함
- 5일 이상 가동 도중 서버가 다운되거나 메모리 사용량이 크게 증가하는 등의 문제가 없어야 함

테스트 결과 오류가 없었음

- 가동 약 5시간 후의 서버 상태:



- 가동 약 5일 후의 서버 상태:



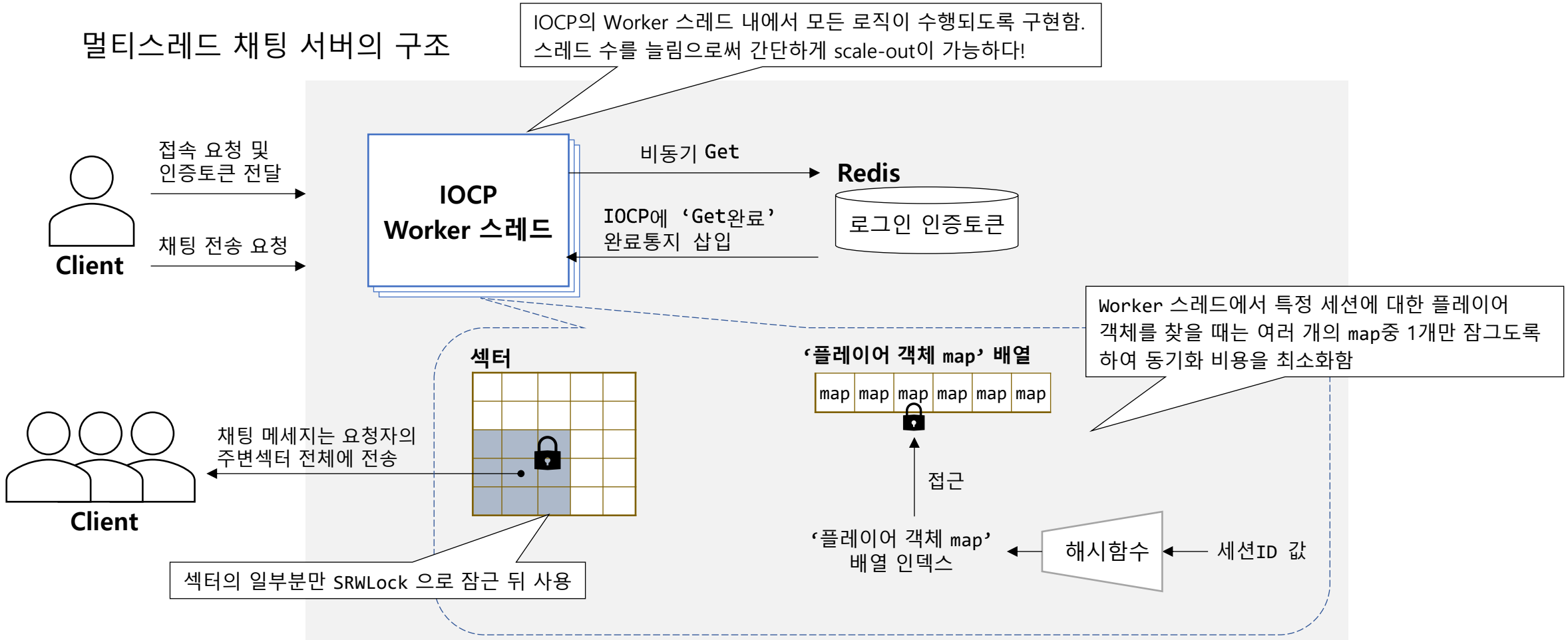
서버 상태를 확인한 결과 게임 콘텐츠의 FPS가 떨어진다고거나, 메모리 사용량이 크게 증가하는 등의 문제가 없었음

1. MMO 게임 서버

APPENDIX : 멀티스레드 채팅 서버 개발

- 멀티스레드 채팅 서버를 개발하여 싱글스레드 채팅 서버와의 성능 비교를 수행함.

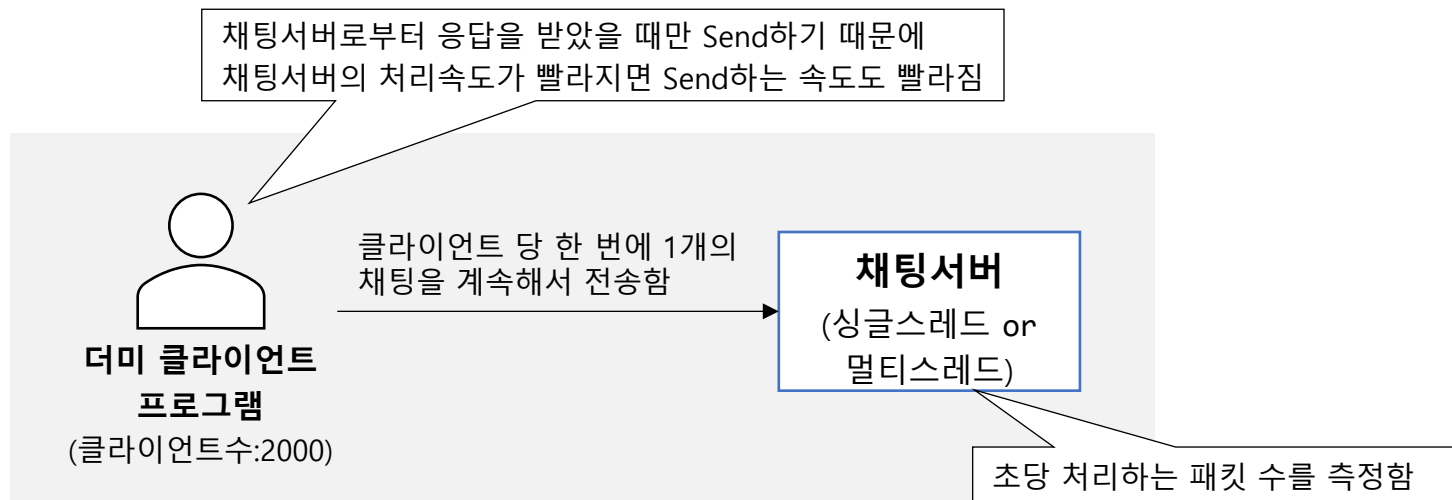
멀티스레드 채팅 서버의 구조



1. MMO 게임 서버

APPENDIX : 멀티스레드 채팅 서버 개발

- 싱글스레드 채팅서버와 멀티스레드 채팅서버의 속도 비교:



- 속도 비교 결과:

서버	초당 처리하는 패킷 수	서버의 CPU 사용률	사용 CPU : Intel Core i5-4460 3.20GHz
싱글스레드 채팅서버	36,109	44.9%	
멀티스레드 채팅서버	73,152	71.7%	

멀티스레드 채팅서버는 싱글스레드 채팅서버에 비해 CPU를 1.60배 더 소모하여 2.02배 더 빠르게 패킷을 처리함

2. IOCP 네트워크 라이브러리

개발환경

- Windows, C++

개발기간

- 2023.01 ~ 2023.02

개요

- 서버 개발을 위한 멀티스레드 기반의 네트워크 라이브러리.
- IOCP를 사용하여 수만개의 TCP 연결을 안정적으로 처리할 수 있도록 구현함.
- 멀티스레드 프로그래밍 연습을 위해 공유자원을 사용할 때 동기화 객체대신 Interlock 계열 함수를 사용하였음.
- 미묘한 멀티스레드 버그의 디버깅을 위해 메모리에 로그를 기록하여 분석함.
- 네트워크 오류의 디버깅을 위해 WireShark 패킷캡처 프로그램을 사용하여 분석함.
- 개발된 네트워크 라이브러리를 사용한 채팅서버를 5일이상 가동하여 테스트함.

2. IOCP 네트워크 라이브러리

네트워크 라이브러리의 사용법

사용자는 라이브러리의 네트워크 클래스를 상속받은 클래스를 만들고, 네트워크 클래스 내의 순수 가상함수를 구현해야 한다.

사용자 Part

라이브러리에서 제공하는
CNetServer 클래스를 상속받음

```
/* 네트워크 클래스를 상속받은 게임서버 클래스 */
class CGameServer : public CNetServer
{
private:
    /* 네트워크 클래스가 요구하는 콜백 가상함수를 구현함 */
    virtual bool OnRecv(__int64 sessionId, CPacket& packet);
    virtual bool OnClientJoin(__int64 sessionId);
    virtual bool OnClientLeave(__int64 sessionId);
};
```

클라이언트 접속, 데이터 수신 등의 네트워크 이벤트 발생 시
라이브러리에서 아래 콜백 함수들을 호출해준다.

- OnClientJoin
새로 접속한 유저를 서버에 등록하는 등의 코드를 작성
- OnRecv
수신된 데이터를 처리하는 콘텐츠 로직 코드를 작성
- OnClientLeave
접속이 종료된 유저를 서버에서 삭제하는 등의 코드를 작성

```
int main()
{
    // 네트워크 객체 생성 및 네트워크 통신 시작
    CGameServer gameServer;
    gameServer.Startup(L"0.0.0.0", 55555);
}
```

사용자는 네트워크 객체를 생성하고, Startup 함수로 네트워크 통신을 시작한다.
Startup 함수를 호출하면 라이브러리 내에서 IOCP가 가동된다.

```
// 세션에 패킷 전송
gameServer.SendPacket(pPlayer->_sessionId, pPacket);

// 세션 연결 끊기
gameServer.Disconnect(pPlayer->_sessionId);
```

SendPacket 함수로 세션에 패킷 데이터를 전송하고,
Disconnect 함수로 원하는 때에 세션의 연결을 끊는다.

2. IOCP 네트워크 라이브러리

네트워크 라이브러리의 기본 동작

라이브러리 Part

Accept 스레드

1. Accept
2. 소켓을 IOCP에 연결
3. 세션 할당
4. **OnClientJoin** 콜백 함수 호출
5. Recv

Alloc

세션 배열



Associate

IOCP

Get 완료통지

Worker 스레드

1. Recv I/O 완료통지 처리
 - 1) 수신한 패킷 데이터의 무결성 검증
 - 2) **OnRecv** 콜백 함수로 사용자에게 데이터 전달
 - 3) Recv
2. Send I/O 완료통지 처리
 - 1) 세션 송신버퍼의 데이터를 계속해서 Send
3. 세션을 닫을 때
 - 1) 소켓을 닫고 세션을 반환함
 - 2) **OnClientLeave** 콜백 함수 호출

사용자 Part

사용자가 구현하는 콜백 가상함수

- OnClientJoin
새로 접속한 유저를 서버에 등록하는 등의 코드를 작성
- OnRecv
수신된 데이터를 처리하는 콘텐츠 로직 코드를 작성
- OnClientLeave
접속이 종료된 유저를 서버에서 삭제하는 등의 코드를 작성

2. IOCP 네트워크 라이브러리

네트워크 라이브러리의 지연된 소켓닫기 스레드

스레드 도입배경:

- 사용자가 "데이터를 보내고 연결끊기" 함수를 호출했을 경우 데이터가 Send된 다음 소켓이 닫히는것을 보장하기 위해 도입함.

라이브러리 Part

Worker 스레드

1. Send I/O 완료통지 처리
 - 1) 세션에 대해 "데이터를 보내고 연결끊기" 함수가 호출되었음을 감지함
 - 2) 지연된 소켓닫기 스레드에게 소켓을 전달함

소켓

메시지큐

소켓

지연된 소켓닫기 스레드

소켓 송신버퍼 내의 데이터가 전송되기를 최대 5ms까지 기다린 후 소켓을 닫는다.

사용자 Part

라이브러리가 제공하는 사용자 호출 함수

- SendPacketAndDisconnect(데이터를 보내고 연결끊기)
세션의 소켓에 SO_LINGER 옵션을 사용하여 5ms의 timeout을 지정한다음 Send 한다.

2. IOCP 네트워크 라이브러리

네트워크 라이브러리의 트래픽 혼잡제어 스레드

스레드 도입배경:

- 네트워크의 상태가 좋지 않을 경우 유실되는 패킷이 대량으로 발생하고, 이로 인해 TCP가 대량의 패킷 재전송을 시도하여 순간적으로 트래픽이 급증함. 이 때 네트워크 상태가 더욱 악화되는것을 막기 위해 트래픽을 억제하는 기능을 도입하였음.

라이브러리 Part

트래픽 혼잡제어 스레드

```
if( '재전송되는 TCP Segment 수' ≥  
    '전체 TCP Segment 수' / 10 )  
{  
    0.1초 동안 트래픽 혼잡 플래그 ON  
}
```

Worker 스레드

- "Send 대기중" 완료통지를 받음
 - 트래픽 혼잡 플래그가 ON 일 경우:
IOCP에 'Send 대기중' 완료통지 삽입
 - 트래픽 혼잡 플래그가 OFF 일 경우:
Send

IOCP

'Send 대기중'
완료통지

사용자 Part

라이브러리가 제공하는 사용자 호출 함수

- SendPacket

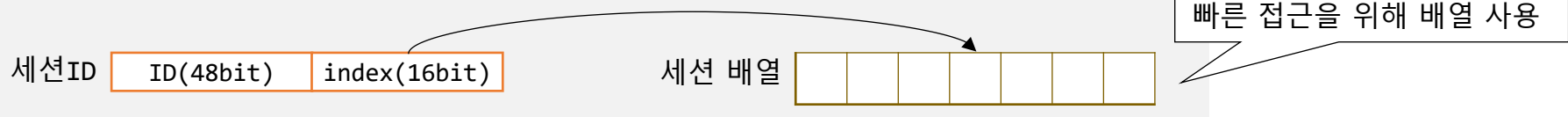
트래픽 혼잡 플래그가 ON 일 경우 Send 하지않고 IOCP에 "Send 대기중" 완료통지를 삽입한다.
트래픽 혼잡 플래그가 OFF 일 경우 Send 한다.

'Send 대기중'
완료통지

2. IOCP 네트워크 라이브러리

Interlock 계열 함수를 사용한 안전한 공유자원(세션) 접근

- 네트워크 라이브러리는 세션ID를 통해 원하는 세션을 배열에서 한번에 찾는다:



- 멀티스레드 환경에서 세션에 안전하게 접근하기위해 다음 조건이 필요:

- 조건1. 내 스레드가 세션 객체를 사용하는 도중에 다른 스레드에서 세션을 닫거나 재할당하지 않아야함
- 조건2. 내 스레드가 찾은 세션이 이미 닫혔거나, 다른 세션으로 재할당 되었다면 접근하지 않아야함

- 조건을 만족시키기 위해 'Use Count & Release Flag' 멤버를 사용함

'Use Count & Release Flag'(64bit) Use Count(int) Release Flag(bool)

세션을 사용중인곳이 아무데도 없는('Use Count' == 0) **동시에**, 세션이 아직 닫히지 않은('Release Flag' == 0) 경우에만 Release Flag 를 1(true)로 변경하고 세션을 닫는다.

- 안전한 세션사용 절차:

세션에 접근할 때

- 1) 세션ID로 배열에서 세션을 찾음
- 2) `InterlockIncrement('Use Count')`
- 3) `if('Release Flag' == true 또는 세션ID 불일치)`
 세션 사용을 끝나치고 종료
- 4) 세션 사용

세션사용을 끝마쳤을 때

- 1) `InterlockDecrement('Use Count')`
- 2) `if(InterlockedCompareExchange64('Use Count & Release Flag', 1, 0) == 0)`
 세션을 닫는다

세션이 이미 닫혔거나, 다른 ID로 재사용됨

2. IOCP 네트워크 라이브러리

디버깅 : 닫히지 않는 세션이 존재하는 버그 (1/2)

- 버그 현상:

모든 클라이언트의 연결을 끊었는데 서버에서 닫히지 않은 세션이 1개 남음

- 버그의 원인 파악을 위해 메모리에 로그를 기록함

파일에 로그를 기록하면 로그 기록에 시간이 너무 오래 걸려서 미묘한 멀티스레드 버그가 잘 발생하지 않기 때문에 메모리에 로그를 기록

메모리 로그 기록 코드 예시▼(WSARecv 함수 호출 전 로그 기록)

```
#ifndef NET_ENABLE_MEMORY_LOGGING
    __int64 logNum = InterlockedIncrement64(&_logNum);    // 로그 번호 얻기
    int logArrayIndex = (int)(logNum & _sizeLogBuffer);    // 로그를 기록할 배열 인덱스 얻기
    MemLog& memLog = _logBuffer[logArrayIndex];
    memLog.logNum = logNum;                                // 로그 기록 시작
    memLog.eLogType = ELogType::WSARecvStart;
    memLog.sessionId = pSession->_sessionId;
    memLog.data1 = (INT64)pSession;
    memLog.data2 = pSession->_ioCount;
    memLog.data3 = pSession->_sock;
#endif

    int retRecv = WSARecv(pSession->_sock, WSABuf, 2, NULL, &flag, (OVERLAPPED*)&pSession->_recvOverlapped, NULL);
```

메모리에 기록한 로그 예시▼(메모리값 1줄 당 1개의 로그를 나타냄):

[illegible]

2. IOCP 네트워크 라이브러리

디버깅 : 닫히지 않는 세션이 존재하는 버그 (2/2)

- 버그 원인 파악:

스레드1

1. 세션A 에서 Recv 하려고함
recv 함수를 호출하기 직전임
세션A 사용카운트=2
4. 세션A에서 recv 함수가 호출됨 (3)
(1) 에서 세션A의 소켓을 닫았지만, 곧바로
재사용되어 유효한 소켓이됨
결과적으로 세션B의 소켓에서 (2)와 (3)의
recv가 호출됨
6. 세션B에 대한 Recv 실패 완료통지를 받음
세션B의 사용카운트 1 감소 → 사용카운트가
0이 되어 세션이 닫힘
하지만 세션A에 대한 Recv 실패 완료통지는
절대로 도착하지 않음 → 세션이 닫히지 않음

스레드2

2. 세션A에서 문제가 발견되어 연결 끊기를 시도함
세션A의 소켓을 닫음 → 소켓이 반환됨 (1)
세션A 사용카운트 1 감소 → 사용카운트 1이
남아있어서 세션이 닫히지는 않음
3. 새로운 클라이언트가 연결됨
소켓 할당 → (1) 에서 반환된 소켓이 재할당됨
세션B 할당, 사용카운트=1
recv (2)

클라이언트

5. 모든 클라이언트가 연결을 끊음

• 해결책:

클라이언트와의 연결을 끊고싶을 때 소켓을 닫지 않고 CancelIoEx 함수로 소켓에 걸린 I/O를 취소만 한다.
그런다음 세션의 사용카운트가 0이 되어 세션을 닫을 때 소켓을 닫는다.

2. IOCP 네트워크 라이브러리

디버깅 : ERROR_SEM_TIMEOUT 네트워크 오류 (1/2)

- 오류 현상:

서버가 매우 많은 수의 클라이언트(15000개 이상의 TCP Connection)와 통신할 때 일부 소켓에서 때때로 ERROR_SEM_TIMEOUT 오류가 발생하면서 연결이 끊김.

- 오류 현상 파악을 위해 WireShark 로 패킷캡처

서버=10.0.2.1::12001, 클라이언트=10.0.2.2::61857

tcp.port == 61857						
No.	Time	Source	Destination		Protocol	Length Info
85750	53.691289	10.0.2.2	10.0.2.1	1.	TCP	133 61857 → 12001 [PSH, ACK, CWR] Seq=179 Ack=5066 Win=2102016 Len=79
86054	53.756466	10.0.2.1	10.0.2.2	2.	TCP	54 12001 → 61857 [ACK] Seq=6526 Ack=258 Win=2102272 Len=0
86286	53.776278	10.0.2.2	10.0.2.1	3.	TCP	133 [TCP Spurious Retransmission] 61857 → 12001 [PSH, ACK] Seq=179 Ack=5066 Win=2102016 Len=79
86289	53.776318	10.0.2.1	10.0.2.2	4.	TCP	66 12001 → 61857 [ACK] Seq=6526 Ack=258 Win=2102272 Len=0 SLE=179 SRE=258
86479	53.975919	10.0.2.2	10.0.2.1	3.	TCP	133 [TCP Spurious Retransmission] 61857 → 12001 [PSH, ACK] Seq=179 Ack=5066 Win=2102016 Len=79
86480	53.975961	10.0.2.1	10.0.2.2	4.	TCP	66 12001 → 61857 [ACK] Seq=6526 Ack=258 Win=2102272 Len=0 SLE=179 SRE=258
87022	54.175954	10.0.2.2	10.0.2.1	3.	TCP	133 [TCP Spurious Retransmission] 61857 → 12001 [PSH, ACK] Seq=179 Ack=5066 Win=2102016 Len=79
87025	54.176132	10.0.2.1	10.0.2.2	4.	TCP	66 12001 → 61857 [ACK] Seq=6526 Ack=258 Win=2102272 Len=0 SLE=179 SRE=258
87958	54.576317	10.0.2.2	10.0.2.1	3.	TCP	133 [TCP Spurious Retransmission] 61857 → 12001 [PSH, ACK] Seq=179 Ack=5066 Win=2102016 Len=79
87961	54.576459	10.0.2.1	10.0.2.2	4.	TCP	66 12001 → 61857 [ACK] Seq=6526 Ack=258 Win=2102272 Len=0 SLE=179 SRE=258
88741	54.855790	10.0.2.1	10.0.2.2	5.	TCP	1514 [TCP Retransmission] 12001 → 61857 [PSH, ACK] Seq=5066 Ack=258 Win=2102272 Len=1460
93753	57.440336	10.0.2.1	10.0.2.2	5.	TCP	1514 [TCP Retransmission] 12001 → 61857 [PSH, ACK] Seq=5066 Ack=258 Win=2102272 Len=1460
98460	62.581779	10.0.2.1	10.0.2.2	5.	TCP	1514 [TCP Retransmission] 12001 → 61857 [PSH, ACK] Seq=5066 Ack=258 Win=2102272 Len=1460
100411	71.332415	10.0.2.1	10.0.2.2	6.	TCP	54 12001 → 61857 [RST, ACK, CWR] Seq=6526 Ack=258 Win=0 Len=0

- 클라이언트가 Seq=179, Len=79 패킷 전송
- 서버가 ACK=258 응답
- 클라이언트가 서버의 ACK 응답을 받지 못해서(패킷 유실) 계속해서 재전송함
- 서버가 계속해서 ACK를 응답함. 하지만 클라이언트는 받지 못했음(패킷 유실)
- 서버도 보낼 데이터가 있어서 클라이언트에게 전송하였으나 클라이언트로부터 응답이 오지 않아 계속해서 재전송함
- 서버가 대략 15초에 걸쳐 재전송 하다가 응답이 오지 않아 TCP가 강제로 연결을 끊음 → ERROR_SEM_TIMEOUT 오류 발생

2. IOCP 네트워크 라이브러리

디버깅 : ERROR_SEM_TIMEOUT 네트워크 오류 (2/2)

- 오류 현상 파악을 위해 자체적으로 기록한 서버 로그를 분석함

(1) 네트워크 상태가 정상적일때의 서버 로그 ↓

```
967 [2022-12-15 15:37:13.763] INFO 138 [network] session:18772, accept:198151 (1277/s), conn:198151 (1277/s), disconn:179379 (1277/s), recv:10799/s, send:130552/s
968 [2022-12-15 15:37:13.783] INFO 138 [server ] player:18772, account:15090, msg proc:11276/s, msg remain:1123, event wait:736ms, CPU usage [T:57.1%, U:37.7%, K:16.2%]
969 [2022-12-15 15:37:13.784] INFO 138 [disconn] sess lim:0, iocp assoc:0, io err (known:193591, 121:0, unknown:0, 64:179379, 995:0, 1236:0, 10038:0, 10053:0, 10054:0)
970 [2022-12-15 15:37:13.786] INFO 138 [disconn] player lim:0, packet (code:0, len:0, decode:0, type:0), login fail:0, dup:0, account:0, sector:0, timeout (login:0, 64:179379, 995:0, 1236:0, 10038:0, 10053:0, 10054:0)
971 [2022-12-15 15:37:13.786] INFO 138 [pool ] packet:4700 (alloc:1700, used:1125, free:3000), message:5400 (alloc:1900, used:1124, free:3500), player:36200 (alloc:36300, free:36300)
972 [2022-12-15 15:37:13.786] INFO 138 [system ] TCP (segment:158720/s, retrans:6834/s, recv:7.18MB/s, send:22.49MB/s), memory(MB) [System C:3569.53, P:155.78, NP:270]
```

초당 전송되는 TCP segment의 수가 대략 15만, 재전송되는 TCP segment의 수가 대략 7000 이다.

(2) ERROR_SEM_TIMEOUT 오류가 발생할 때의 서버 로그 ↓

```
1023 [2022-12-15 15:37:21.817] INFO 146 [network] session:18966, accept:209308 (1965/s), conn:209308 (1965/s), disconn:190342 (2145/s), recv:11972/s, send:279429/s
1024 [2022-12-15 15:37:21.836] INFO 146 [server ] player:19008, account:15820, msg proc:14117/s, msg remain:0, event wait:641ms, CPU usage [T:93.8%, U:77.3%, K:16.2%]
1025 [2022-12-15 15:37:21.846] INFO 146 [disconn] sess lim:0, iocp assoc:0, io err (known:207261, 121:0, unknown:0, 64:190384, 995:5, 1236:0, 10038:0, 10053:0, 10054:0)
1026 [2022-12-15 15:37:21.847] INFO 146 [disconn] player lim:0, packet (code:0, len:0, decode:0, type:0), login fail:0, dup:5, account:0, sector:0, timeout (login:0, 64:190384, 995:5, 1236:0, 10038:0, 10053:0, 10054:0)
1027 [2022-12-15 15:37:21.847] INFO 146 [pool ] packet:4700 (alloc:1700, used:820, free:3000), message:5400 (alloc:800, used:0, free:4600), player:37100 (alloc:36300, free:36300)
1028 [2022-12-15 15:37:21.850] INFO 146 [system ] TCP (segment:87090/s, retrans:52788/s, recv:8.17MB/s, send:63.21MB/s), memory(MB) [System C:4194.61, P:156.50, NP:893]
```

초당 전송되는 TCP segment의 수가 대략 8만, 재전송되는 TCP segment의 수가 대략 5만 이다.

- 해결책

네트워크 상태가 좋지 않아 유실되는 패킷이 발생할 때, TCP가 대량의 패킷을 재전송하여 네트워크 상태가 심각하게 악화되는것을 확인함.

네트워크 상태가 악화되는 것을 막기 위해 "트래픽 혼잡제어 스레드" 를 추가하여 네트워크가 혼잡할 때 서버에서 잠시동안 Send를 하지 못하도록 막음.

스레드의 기능은 18번 슬라이드 참조

2. IOCP 네트워크 라이브러리

라이브러리 오류 검증 테스트

테스트 조건

- IOCP 네트워크 라이브러리를 사용하여 채팅서버 개발(채팅서버의 구조는 MMO 게임서버의 채팅서버와 동일)
- 더미 클라이언트 프로그램을 사용하여 5000명의 클라이언트 연결
더미 프로그램은 서버와 통신하면서 서버가 잘못된 패킷을 보내지 않았는지, 연결을 잘못 끊지 않았는지 등을 체크함
- 서버에 초당 접속하는 유저 400명, 나가는 유저 400명, 서버가 초당 전송받는 패킷 7000개 수준으로 테스트
- 5일 이상 실행 시 오류가 없어야 함

테스트 결과 오류가 없었음

- 5일 이상 가동 후의 채팅서버 상태:

채팅 서버		서버 실행시간 약 50만초(5.8일)		채팅 서버 총 접속횟수 21억, 초당 접속자수 약 400	채팅 서버가 받는 패킷 초당 7800개, 보내는 패킷 초당 58000개
4040083	[2023-01-10 13:51:16.877]	INFO	505010 [network]	session:4463, accept:217433316 (410/s), conn:217433316 (410/s), disconn:217428853 (394/s), recv:7828/s, send:58341/s, recvComp:8222/s, sendComp:57513/s	
4040084	[2023-01-10 13:51:16.878]	INFO	505010 [server]	player:4464, account:4233, msg proc:6632/s, msg remain:2, event wait:848ms, CPU usage [T:30.3%, U:17.6%, K:12.3%] [Server:12.8%, U:8.7%, K:4.1%]	
4040085	[2023-01-10 13:51:16.878]	INFO	505010 [disconn]	sess lim:0, iocp assoc:0, io err (known:218064538, 121:0, unknown:0), packet (code:0, len:0, decode:0, type:0)	
4040086	[2023-01-10 13:51:16.879]	INFO	505010 [disconn]	player lim:0, login fail:0, dup:104, account:0, sector:0, timeout (login:0, heart:0), redis (no key:0, invalid key:0)	
4040087	[2023-01-10 13:51:16.879]	INFO	505010 [pool]	packet:5800 (alloc:700, used:0, free:5100), message:2300 (alloc:700, used:0, free:1600), player:22400 (alloc:19800, used:4464, free:2600)	
4040088	[2023-01-10 13:51:16.880]	INFO	505010 [DB]	query count:0/s, remain query:0, query run time (avg:0.0ms, max:0.0ms, min:0.0ms), pool (size:0, alloc:0)	
4040089	[2023-01-10 13:51:16.880]	INFO	505010 [system]	TCP (segment:56386/s, retrans:112/s, recv:3.62MB/s, send:9.39MB/s), memory(MB) [System C:3741.95, P:195.04, NP:272.48] [Process C:220.30, P:1.02, NP:3.02]	
4040090	[2023-01-10 13:51:16.881]	INFO	505010 [etc]	GQCS wait time(ms): (? , 922, ? , 918), traffic control:10374, error:0	

3. 게임 콘텐츠 전용 IOCP 네트워크 라이브러리

개발환경

- Windows, C++

개발기간

- 2023.03 ~ 2023.03

개요

- 고정된 FPS를 가지는 게임 콘텐츠 개발을 위한 IOCP 네트워크 라이브러리.
- IOCP를 사용하여 수만개의 TCP 연결을 안정적으로 처리할 수 있도록 구현함.
- 사용자는 라이브러리가 제공하는 콘텐츠 클래스를 상속받아 고정된 FPS를 가지는 콘텐츠를 개발한다.
- 콘텐츠 객체마다 자신에게 속한 세션을 관리하며, 세션의 패킷은 반드시 세션이 속한 콘텐츠에서만 처리된다.
- 네트워크 라이브러리의 오류 검증을 위해 에코서버를 만들어 테스트함.

3. 게임 콘텐츠 전용 IOCP 네트워크 라이브러리

네트워크 라이브러리의 사용법

사용자는 라이브러리의 네트워크 클래스, 콘텐츠 클래스를 상속받은 클래스를 만들고, 콘텐츠 클래스 내의 순수 가상함수를 구현해야 한다.

사용자 Part

```
/* 네트워크 클래스를 상속받은 게임서버 클래스 */  
class CGameServer : public CNetServer  
{
```

게임 콘텐츠 클래스는 실질적인 게임 로직을 수행하는 주체이다.
게임에 존재하는 콘텐츠마다 서로다른 게임 콘텐츠 클래스를 작성해야 한다.

```
/* 콘텐츠 클래스를 상속받은 게임 콘텐츠 클래스 */  
class CGameContents1 : public CContents  
{  
private:  
    /* 콘텐츠 클래스가 요구하는 콜백 가상함수를 구현함 */  
    virtual void OnRecv(__int64 sessionId, CPacket& packet);  
    virtual void OnSessionJoin(__int64 sessionId, PVOID data);  
    virtual void OnSessionLeave(__int64 sessionId);  
    virtual void OnUpdate();  
};
```

데이터 수신 등의 네트워크 이벤트가 발생할 때 마다 라이브러리가 호출해줌.

- OnRecv : 수신된 패킷 데이터를 처리하는 코드를 작성
- OnSessionJoin : 콘텐츠에 새로 진입한 유저에 대한 초기화 코드를 작성
- OnSessionLeave : 접속이 종료된 유저에 대한 정리 코드를 작성
- OnUpdate : 게임 콘텐츠의 프레임 업데이트 로직 작성

```
int main()  
{  
    // 게임 서버 및 게임 콘텐츠 생성  
    CGameServer gameServer;  
    CGameContents1 contents1;  
    CGameContents2 contents2;  
  
    // 게임 콘텐츠를 서버에 Attach 한다.  
    gameServer.AttachContents(&contents1);  
    gameServer.AttachContents(&contents2);  
  
    // 클라이언트가 최초로 접속하는 콘텐츠를 지정한다.  
    gameServer.SetInitialContents(&contents1);  
  
    // 네트워크 통신을 시작한다.  
    gameServer.Startup(L"0.0.0.0", 55555);
```

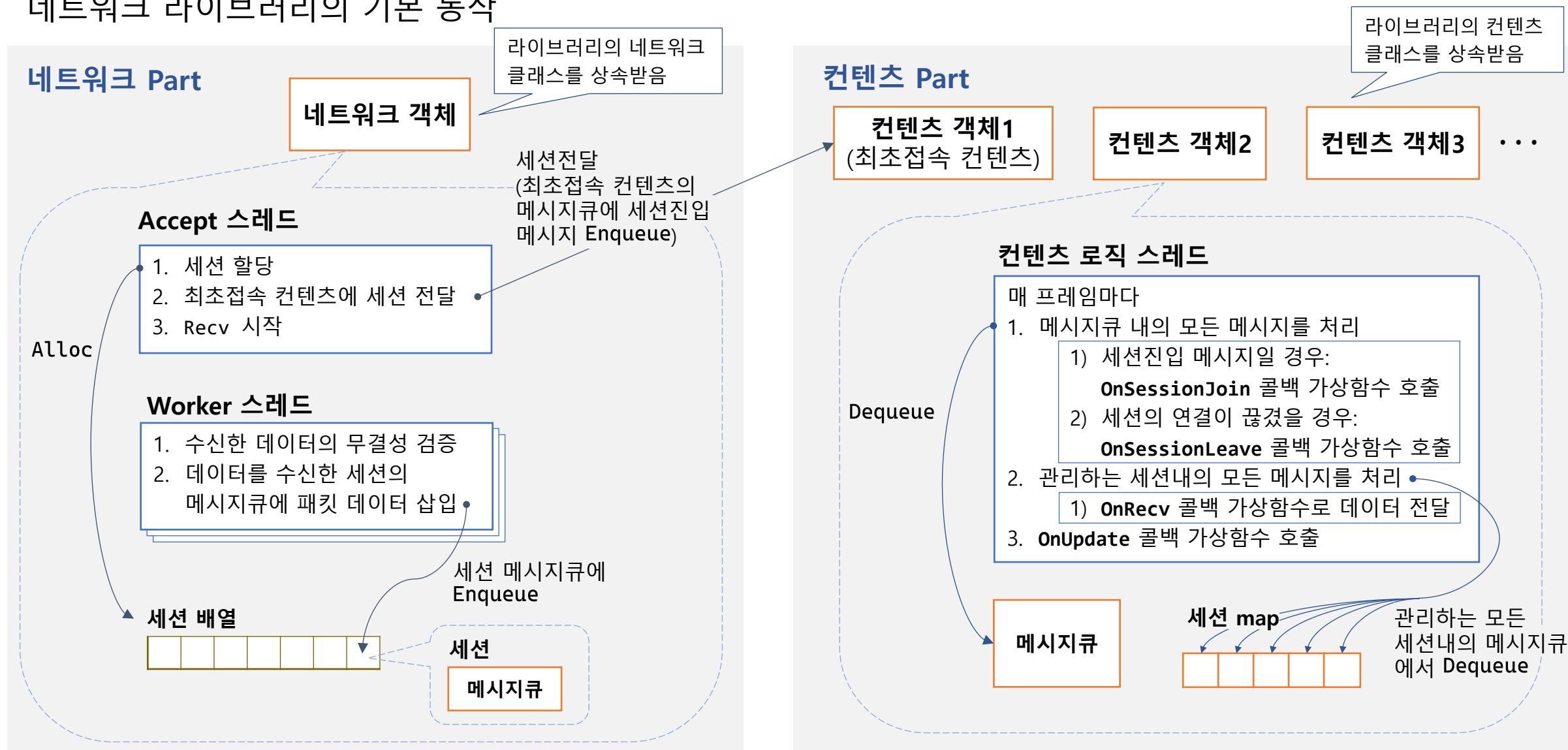
콘텐츠 객체를 라이브러리에 Attach 하면 라이브러리가
콘텐츠 객체 내의 로직 스레드를 실행해준다.

클라이언트가 게임서버에 처음 접속하면
라이브러리는 최초접속 콘텐츠에 세션을 넣어준다.

IOCP를 가동하고 네트워크 통신을 시작한다.

3. 게임 콘텐츠 전용 IOCP 네트워크 라이브러리

네트워크 라이브러리의 기본 동작



3. 게임 콘텐츠 전용 IOCP 네트워크 라이브러리

에코서버를 만들어 라이브러리를 테스트함

- 라이브러리 오류 검증을 위해 에코서버를 개발하여 테스트함

더미 클라이언트
프로그램

계속해서 패킷 전송,
연결끊기 및 재연결 수행

받은 데이터를 그대로 전송

에코서버

접속 승인 후 에코
컨텐츠로 세션 이동

인증 콘텐츠
(최초접속 콘텐츠)

에코 콘텐츠1

에코 콘텐츠2

에코 콘텐츠3

세션을 랜덤하게 다른
컨텐츠로 이동시킴

테스트 결과

- 7일 이상 에코서버를 실행하였으나 아무런 문제가 발생하지 않았음

```
C:\Wproject\GameEchoS
72만초(8.3일) 동안 실행
초당 1600번 accept
받는 패킷 초당 30만개, 보내는 패킷 초당 30만개
7218723 [2023-01-19 00:04:41.887] INFO 721872 [network] session:4807, accept:1269403213 (1632/s), conn:1269403213 (1632/s), disconn:1269398406 (1659/s), recv:308911/s, send:308965/s, recvComp:8649/s, sendComp:44033/s
7218724 [2023-01-19 00:04:41.888] INFO 721872 [game] CPU usage [T:23.4%, U:5.7%, K:15.7%] [Server:18.5%, U:5.6%, K:12.8%]
7218725 [2023-01-19 00:04:41.888] INFO 721872 [auth] player:913, msg proc:1741/s (internal:1667/s), FPS:50 (avg1m:50, max:51, min:42), DT:0.020 (max:0.095, min:0.000)
7218726 [2023-01-19 00:04:41.889] INFO 721872 [echo] player:1294, msg proc:100872/s (internal:884/s), FPS:50 (avg1m:50, max:51, min:38), DT:0.020 (max:0.117, min:0.005)
7218727 [2023-01-19 00:04:41.889] INFO 721872 [echo2] player:1293, msg proc:103656/s (internal:896/s), FPS:50 (avg1m:49, max:51, min:38), DT:0.020 (max:0.122, min:0.003)
7218728 [2023-01-19 00:04:41.890] INFO 721872 [echo3] player:1305, msg proc:102696/s (internal:880/s), FPS:49 (avg1m:50, max:51, min:38), DT:0.020 (max:0.108, min:0.001)
7218729 [2023-01-19 00:04:41.890] INFO 721872 [disconn] sess lim:0, by normal:0, packet (code:0, len:0, decode:0)
7218730 [2023-01-19 00:04:41.890] INFO 721872 [disconn] plyr lim:0, by normal:0, packet (code:0, len:0, decode:0)
7218731 [2023-01-19 00:04:41.891] INFO 721872 [pool] packet:113000 (alloc:30000, used:90, free:6000), message:1300 (alloc:400, used:2, free:6000), player:30900 (alloc:20700, used:4807, free:10200)
7218732 [2023-01-19 00:04:41.891] INFO 721872 [system] TCP (segment:18731/s, retrans:1/s, recv:8.44MB/s, send:7.94MB/s), memory(MB) [System C:3529.91, P:220.68, NP:196.70] [Process C:245.32, P:1.06, NP:3.88]
```

3. 게임 콘텐츠 전용 IOCP 네트워크 라이브러리

APPENDIX : 속도 프로파일링을 통한 성능 개선 (1/2)

- MMO 게임서버 개발 도중, 대량의 더미 플레이어를 투입했을 때 게임 콘텐츠의 프레임이 적정 수준으로 유지되지 않는 문제가 발생함
- 자체 개발한 속도 프로파일러를 사용하여 주요 함수들의 속도를 측정함

1분동안의 속도 프로파일링 결과 (호출횟수 기준으로 내림차순 정렬)

함수	평균소요시간(μ s)	호출횟수
SendAroundSector	121	134404
MonsterFindTarget	6	69235
PacketProc_StopCharacter	183	41856
PacketProc_Pick	156	27297
MovePlayerTileAndSector	67	21188
PacketProc_Attack2	171	13565
MonsterAttack	240	9144

SendAroundSector 함수의
호출 횟수가 가장 많음

- 호출 횟수가 가장 많은 SendAroundSector 함수의 속도가 느린 이유를 분석함:

SendAroundSector 함수는 특정 섹터를 중심으로 11 x 11 섹터내의 모든 플레이어에게 1개의 패킷을 전송하는 함수였음
그런데 패킷 1개를 전송하려 할 때 마다 실제로 소켓에 Send 하는것이 서버에 부담이 되는것을 파악함

그래서 네트워크 라이브러리에 "매 프레임마다 패킷을 모아서 보내기" 모드를 추가함

"매 프레임마다 패킷을 모아서 보내기" 모드를 적용하면 콘텐츠 코드에서 패킷을 Send하면 소켓에 즉시 Send 하지 않고,
패킷을 모아두었다가 1프레임이 종료될 때 소켓에 한번에 Send함

3. 게임 콘텐츠 전용 IOCP 네트워크 라이브러리

APPENDIX : 속도 프로파일링을 통한 성능 개선 (2/2)

- “매 프레임마다 패킷을 모아서 보내기” 모드를 적용했을 때의 속도 프로파일링 결과:

1분동안의 속도 프로파일링 결과 (호출횟수 기준으로 내림차순 정렬)

함수	평균소요시간(μ s) (모드 적용)	평균소요시간(μ s) (모드 미적용)	호출횟수
SendAroundSector	63	121	134404
MonsterFindTarget	5	6	69235
PacketProc_StopCharacter	87	183	41856
PacketProc_Pick	77	156	27297
MovePlayerTileAndSector	29	67	21188
PacketProc_Attack2	87	171	13565
MonsterAttack	153	240	9144

“매 프레임마다 패킷을 모아서 보내기” 모드 적용시 속도가 빨라짐

- 호출 횟수가 가장 많은 SendAroundSector 함수의 속도가 2배 빨라졌고,
SendAroundSector 함수를 사용하는 다른 함수들의 속도도 빨라져 전반적인 서버 성능이 개선됨

4. Tool : 서버 모니터링 클라이언트

개발환경

- C#, Windows Forms

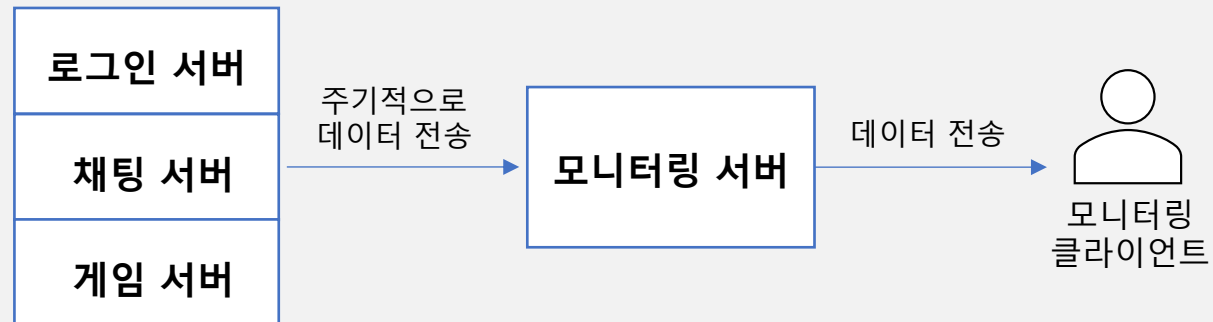
개발기간

- 2023.10 ~ 2023.10

개요

- 게임서버의 상태를 한눈에 파악할 수 있는 모니터링 클라이언트 개발.
- 모니터링 서버로부터 실시간으로 데이터를 전달받아 화면에 그래프로 표시함.
- 빠른 개발을 위해 C# 사용.

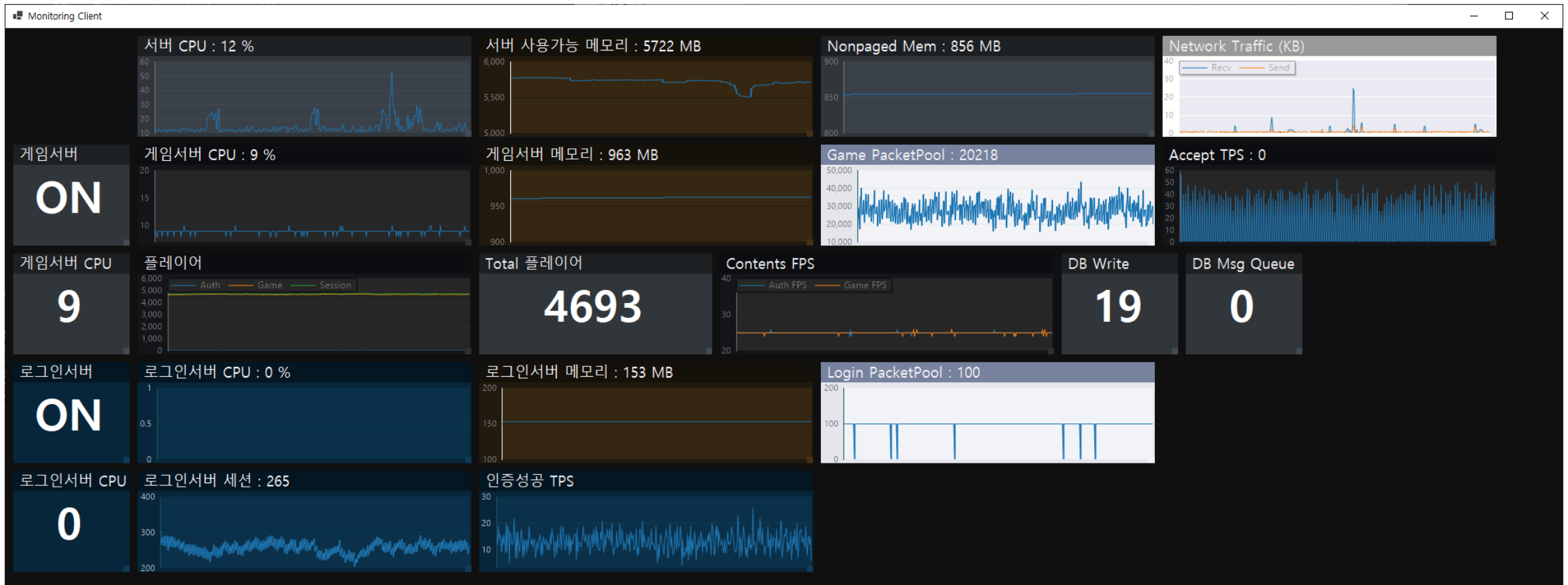
모니터링 데이터 전송 경로



4. Tool : 서버 모니터링 클라이언트

모니터링 클라이언트 Tool 실행화면:

- 모니터링 서버로부터 전달받은 데이터를 실시간 그래프로 표시한다.
- 프로그램 실행 영상 : <https://youtu.be/3XjEC0eIARU>



5. Lockfree Stack

개발환경

- C++

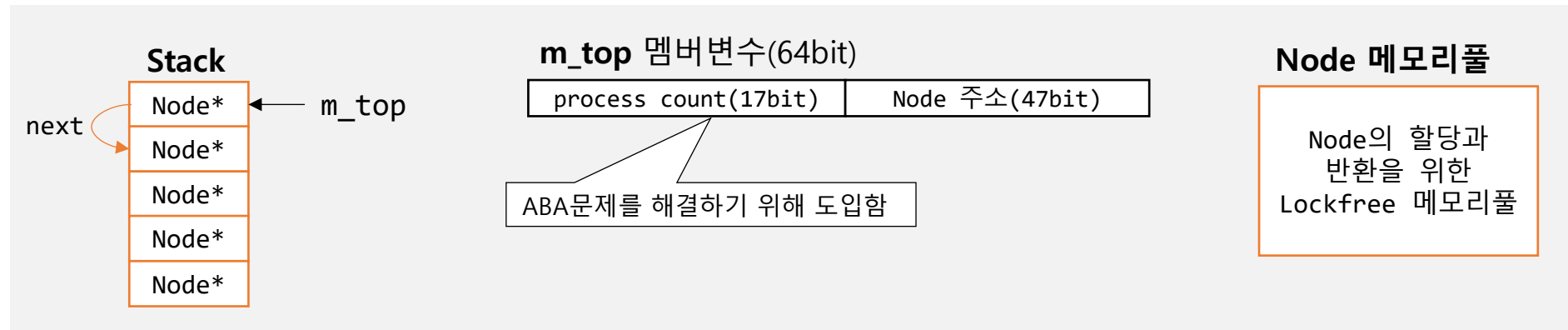
개발기간

- 2022.08 ~ 2022.08

개요

- 멀티스레드 환경에서 정상적으로 작동하는 Lockfree Stack 구현.
- 동기화 객체 없이 Interlock 계열 함수로만 동기화를 수행함.
- 디버깅을 위해 메모리에 로그를 기록하여 분석함.
- 오류 검증을 위해 24시간 테스트 수행.

Lockfree Stack의 멤버



5. Lockfree Stack

Push

1. 값 준비

`old_top = m_top`

process count	top 노드
---------------	--------

`new_top =`

process count + 1	새로 할당받은 노드
-------------------	------------

2. Push

- 1) `InterlockedCompareExchange64(m_top, new_top, old_top)`
- 2) Interlock 함수 실패 시 1번으로 돌아감

`m_top`(스택의 현재 top)이 `old_top`과 같다면(즉, 다른 스레드에서 `m_top`을 변경하지 않았음) `m_top`을 `new_top`으로 변경한다.

Pop

1. 값 준비

`old_top = m_top`

process count	top 노드
---------------	--------

`new_top =`

process count + 1	top노드->next
-------------------	-------------

2. Pop

- 1) `InterlockedCompareExchange64(m_top, new_top, old_top)`
- 2) Interlock 함수 실패 시 1번으로 돌아감
- 3) `old_top`의 Node를 메모리풀에 반환

5. Lockfree Stack

디버깅 : ABA문제 (1/3)

버그현상:

- Pop 로직 마지막에서 Pop한 노드를 delete 할 때 힙 손상 오류가 발생함
(주의: 개발 초기 로직에서 발생한 오류임. m_top 멤버변수가 process count를 포함하지 않았고, 노드의 할당과 해제에 new, delete를 사용하였음)

버그의 원인 파악을 위해 메모리에 로그를 기록함:

- Push와 Pop 성공 직후(**InterlockedCompareExchange64** 함수 성공 직후) 로그를 기록함.
- 메모리 로그 기록 코드 ▼

```
    } while (InterlockedCompareExchange64((__int64*)&_topId, (__int64)newNodeId, (__int64)_topId) != (__int64)_topId); // Push

#ifdef STACK_ENABLE_LOGGING
    // Interlock 함수로 Push를 성공한 직후 메모리에 로그를 기록한다.
    __int64 logBufferIndex = InterlockedIncrement64(&_logBufferIndex); // 로그 번호 얻기
    int index = (int)(logBufferIndex & (__int64)_sizeLogBuffer); // 로그를 기록할 배열 인덱스 얻기
    _logBuffer[index].index = logBufferIndex; // 로그 배열에 로그 번호 기록
    _logBuffer[index].threadId = GetCurrentThreadId(); // 스레드ID 기록
    _logBuffer[index].workType = eWorkType::PUSH; // 작업 유형(Push) 기록
    _logBuffer[index].topNode = pNewNode; // 현재 top 노드 주소 기록
    _logBuffer[index].topNextNode = pNewNode->pNext; // 현재 top->next 노드 주소 기록
    _logBuffer[index].otherNode = pTop == nullptr ? nullptr : pTop->pNext; // 이전 top 노드 주소 기록
#endif
```

5. Lockfree Stack

디버깅 : ABA문제 (2/3)

메모리에 기록한 로그를 보기 쉽게 텍스트로 변환함:

	로그번호	작업유형	스레드ID	old_top 주소 (delete됨여부)	new_top 주소 (new됨여부)
(1)	1158072	POP	5696	00000293E52A6B00 (deleted)	00000293E52AA560
	1158073	POP	7968	00000293E52AA560 (deleted)	00000293E52A7020
(2)	1158074	POP	7968	00000293E52A7020 (deleted)	00000293E52AA660
	1158075	PUSH	7968	00000293E52AA660	00000293E52AA560 (new)
	1158076	PUSH	7968	00000293E52AA560	00000293E52AA6C0 (new)
	1158077	POP	7968	00000293E52AA6C0 (deleted)	00000293E52AA560
	1158078	POP	7968	00000293E52AA560 (deleted)	00000293E52AA660
	1158079	PUSH	7968	00000293E52AA660	00000293E52AA620 (new)
	1158080	PUSH	7968	00000293E52AA620	00000293E52AA5E0 (new)
	1158081	POP	7968	00000293E52AA5E0 (deleted)	00000293E52AA620
	1158082	POP	7968	00000293E52AA620 (deleted)	00000293E52AA660
	1158083	PUSH	7968	00000293E52AA660	00000293E52AA560 (new)
(4)	1158084	POP	5696	00000293E52AA560 (deleted)	00000293E52A7020
(5)	1158085	POP	7968	00000293E52A7020 (deleted)	00000293E52AA660

이 로그 기록 후 힙 손상 오류 발생

버그 발생 절차:

- (1) 5696 스레드가 POP 한 다음, 한번 더 POP 하기 위해 old_top = 00000293E52AA560, new_top = 00000293E52A7020 으로 저장한다음 잠시 일시정지됨.
- (2) 이후 7968 스레드가 POP을 하면서 5696 스레드가 new_top(00000293E52A7020) 으로 저장해두었던 노드를 delete 하였음.
- (3) 이후 7968 스레드가 PUSH를 하면서 5696 스레드가 old_top(00000293E52AA560) 으로 저장해두었던 노드와 동일한 노드를 다시 스택의 top으로 설정하였음.
- (4) 5696 스레드가 깨어나서 POP을 재개하는데, 스택의 top이 저장해두었던 old_top(00000293E52AA560)과 동일하여서 POP을 성공시켰음. **(ABA문제)**
POP을 성공하여 저장해둔 new_top(00000293E52A7020) 노드를 스택의 top으로 설정하였지만, 이 노드의 주소는 (2)에서 이미 delete된 주소임.
- (5) 이후 7968 스레드가 POP을 한다음 00000293E52A7020 노드를 delete 하였는데, 이 주소는 (2)에서 이미 delete된 주소여서 힙 손상 오류가 발생하였음.

5. Lockfree Stack

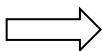
디버깅 : ABA문제 (3/3)

해결책:

- 문제의 근본적인 원인은 내 스레드가 스택의 top 노드 주소를 조사한 다음 다른 스레드가 스택의 top을 변경하였는데, 우연히 스택의 현재 top 노드 주소가 내가 조사했던 top 노드 주소와 일치한다면 **InterlockedCompareExchange64** 함수가 성공한다는 것임.
- 이 문제를 해결하기 위해 스택의 m_top 멤버변수를 Node* 타입이 아닌, Node* 와 작업 카운트(process count)를 합성한 값으로 변경함.

기존 **m_top** 멤버변수(64bit)

Node 주소(64bit)



변경된 **m_top** 멤버변수(64bit)

process count(17bit)	Node 주소(47bit)
----------------------	----------------

이 값은 Pop 또는 Push를 할 때 마다 1씩 증가한다. 따라서 top 노드의 주소가 우연히 이전에 top이었던 노드 주소와 같아지더라도, process count는 다르기 때문에 **InterlockedCompareExchange64** 함수가 실패한다.

5. Lockfree Stack

오류검증 테스트 수행

테스트 조건:

- 하나의 Lockfree Stack에서 5번 Push 하고 5번 Pop 하는 작업을 4개 스레드에서 동시에 수행함.
- 24시간 이상 오류 없이 작동해야함.

테스트 결과:

- 초당 약 670만번의 작업을 27시간 이상 수행하였는데 이상이 없었음

```
C:\project\LockfreeStack\0822_lockfree_stack.exe
[98095 sec] Stack size:8, Node pool size:20, Total run count: 660614458080 (avg:6734375/s), Per thread run count: (168151372899, 165903840437, 160687835851, 165871408893)
[98109 sec] Stack size:12, Node pool size:20, Total run count: 660885652307 (avg:6734330/s), Per thread run count: (168172883448, 165925224514, 160710564308, 165893269301)
[98124 sec] Stack size:7, Node pool size:20, Total run count: 660981940366 (avg:6734315/s), Per thread run count: (168198378289, 165950577114, 160737586254, 165919127666)
[98136 sec] Stack size:10, Node pool size:20, Total run count: 661075752004 (avg:6734300/s), Per thread run count: (168218007793, 165970121862, 160758410420, 165939112232)
[98151 sec] Stack size:6, Node pool size:20, Total run count: 661155876095 (avg:6734286/s), Per thread run count: (168241636615, 165993722474, 160783499043, 165963082234)
[98165 sec] Stack size:5, Node pool size:20, Total run count: 661155876095 (avg:6734286/s), Per thread run count: (168264702547, 166016629385, 160807928253, 165986491819)
[98177 sec] Stack size:5, Node pool size:20, Total run count: 661155876095 (avg:6734286/s), Per thread run count: (168284350421, 166036241513, 160828794403, 166006489758)
```

6. TLS 메모리풀

개발환경

- C++

개발기간

- 2022.09 ~ 2022.09

개요

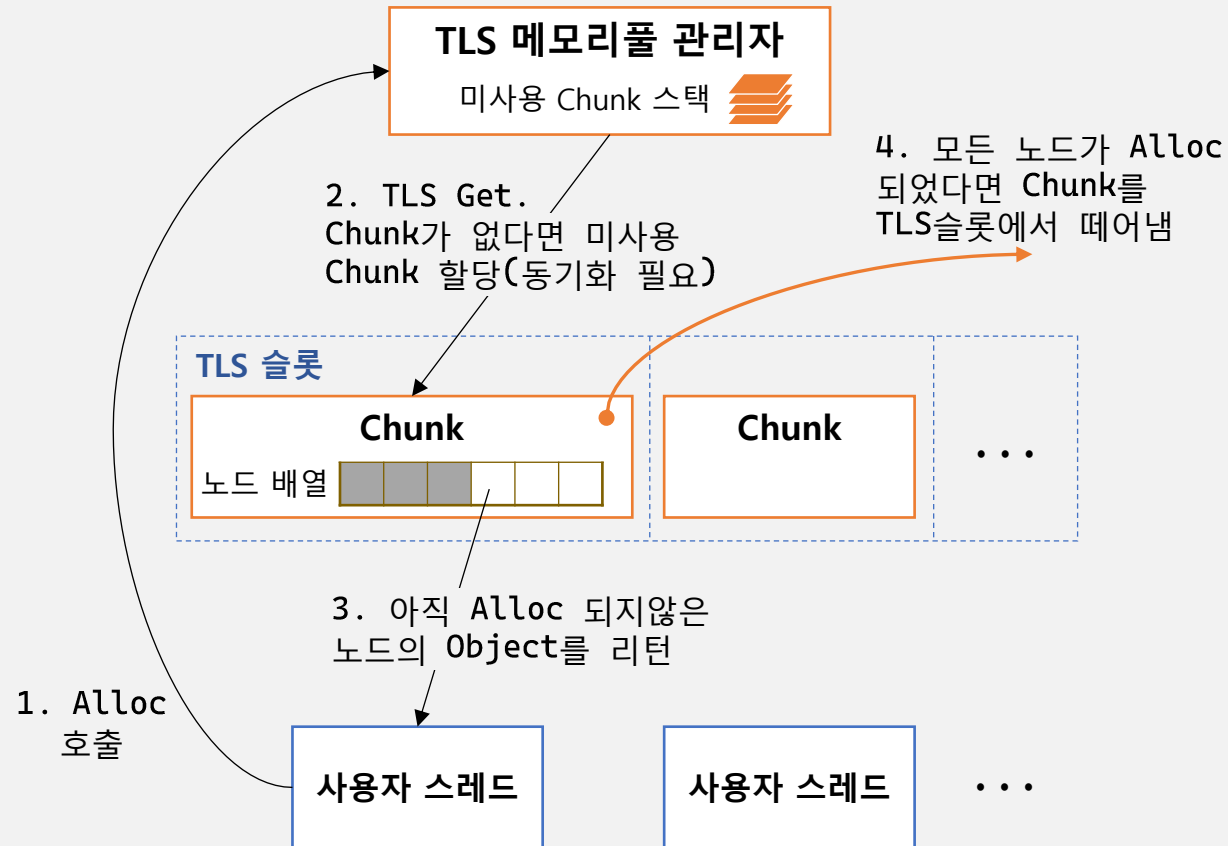
- TLS를 사용하여 동기화 횟수를 최소화한 메모리풀 개발
- 오류 검증을 위한 테스트 수행
- 동적할당과의 속도 비교

6. TLS 메모리풀

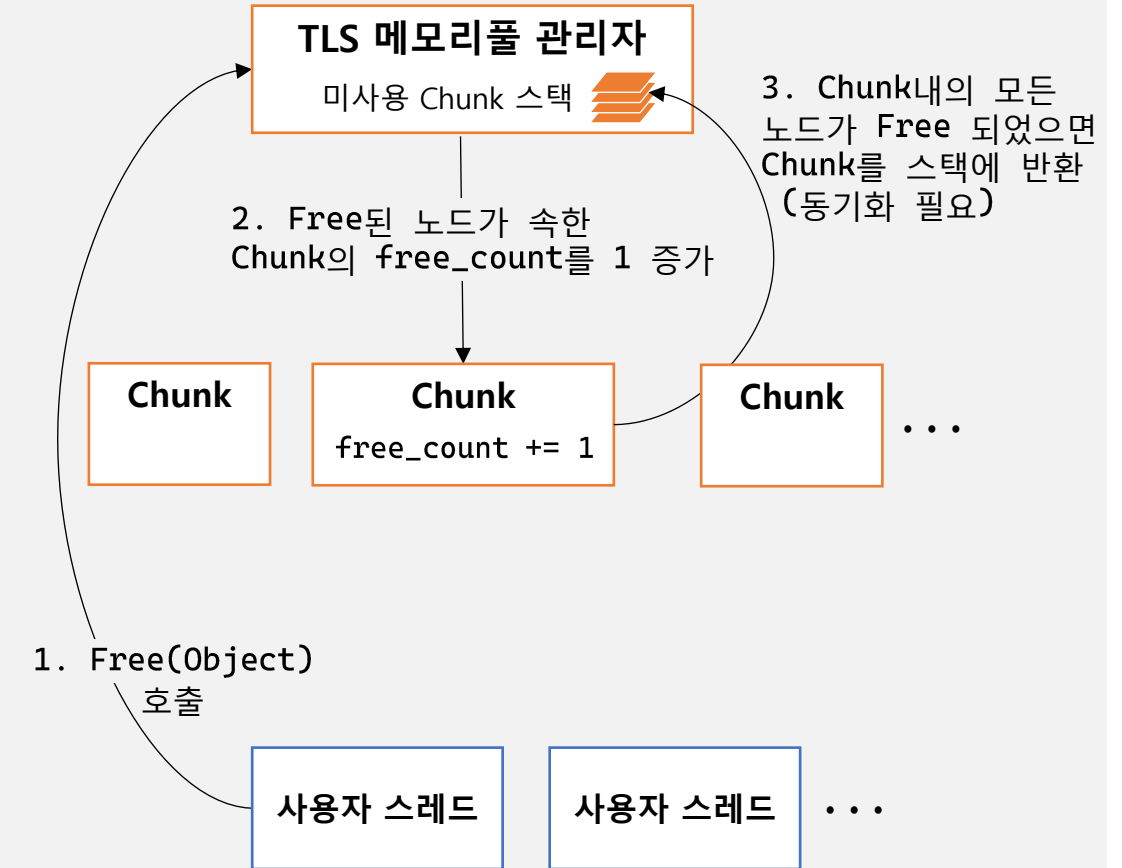
TLS 메모리풀 구조

- TLS 메모리풀 관리자로부터 Chunk를 할당받거나 반환할 때만 동기화를 하기때문에 속도가 빠르다.

Alloc



Free

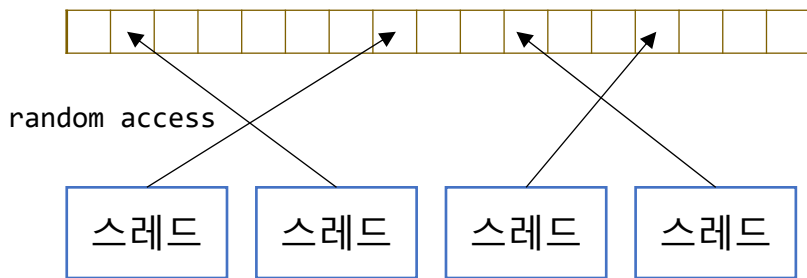


6. TLS 메모리풀

오류 검증을 위한 테스트 수행

테스트 절차:

Object* 배열 (Object 크기:128byte, 배열길이:32767)



4개 스레드가 다음 작업을 반복함:

1. TLS 메모리풀에서 alloc 받은 Object 객체를 배열의 랜덤 위치에 삽입한다.
2. 배열의 랜덤 위치의 객체를 free 한다.

이 과정에서 TLS 메모리풀의 객체 사용량이 지속적으로 증가하거나, 동일 객체가 중복 alloc 또는 중복 free되는 문제가 없어야 함

테스트 결과:

- 초당 약 10,000,000 번 alloc 하고 free 하는것을 24시간 이상 수행하였으나 문제가 없었음
- 실행 15분후의 상태:

```
C:\project\TLSTest\TLSMemoryPool\0919_TLS_mem TLS 메모리풀 내의 객체수: 140,000 프로세스 메모리 사용량: 19.84MB
[run 899 sec] pool state: (size:140000, allocated chunk:854, free chunk:546, allocated object:16438), threads: (alloc object:9942574/s, free object:9942511/s), memory usage:19.84MB
[run 900 sec] pool state: (size:140000, allocated chunk:839, free chunk:561, allocated object:16366), threads: (alloc object:9957954/s, free object:9958029/s), memory usage:19.84MB
[run 901 sec] pool state: (size:140000, allocated chunk:840, free chunk:560, allocated object:16359), threads: (alloc object:9923168/s, free object:9923176/s), memory usage:19.84MB
[run 902 sec] pool state: (size:140000, allocated chunk:849, free chunk:551, allocated object:16433), threads: (alloc object:9945769/s, free object:9945691/s), memory usage:19.84MB
[run 903 sec] pool state: (size:140000, allocated chunk:838, free chunk:562, allocated object:16381), threads: (alloc object:9965067/s, free object:9965117/s), memory usage:19.84MB
```

- 실행 24시간후의 상태:

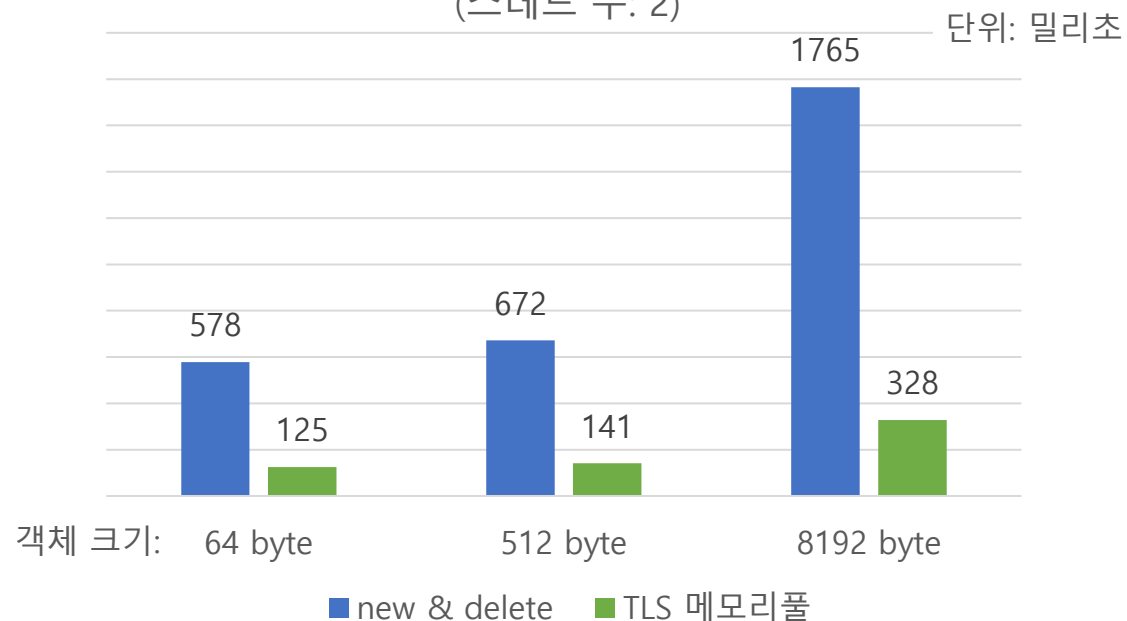
```
C:\project\TLSTest\TLSMemoryPool\0919_TLS_mem TLS 메모리풀 내의 객체수: 140,000 프로세스 메모리 사용량: 19.84MB
[run 87884 sec] pool state: (size:140000, allocated chunk:838, free chunk:562, allocated object:16227), threads: (alloc object:8630584/s, free object:8630723/s), memory usage:19.84MB
[run 87885 sec] pool state: (size:140000, allocated chunk:847, free chunk:553, allocated object:16369), threads: (alloc object:8689297/s, free object:8689162/s), memory usage:19.84MB
[run 87886 sec] pool state: (size:140000, allocated chunk:867, free chunk:533, allocated object:16433), threads: (alloc object:8621343/s, free object:8621276/s), memory usage:19.84MB
[run 87887 sec] pool state: (size:140000, allocated chunk:843, free chunk:557, allocated object:16522), threads: (alloc object:8637592/s, free object:8637502/s), memory usage:19.84MB
[run 87888 sec] pool state: (size:140000, allocated chunk:838, free chunk:562, allocated object:16405), threads: (alloc object:8626733/s, free object:8626849/s), memory usage:19.84MB
```


6. TLS 메모리풀

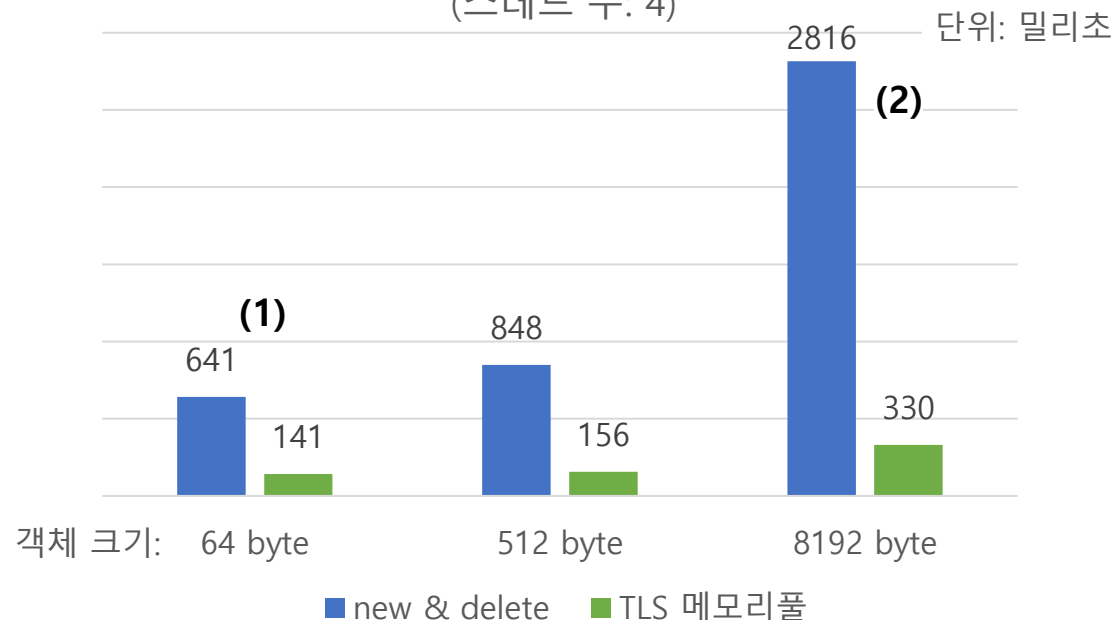
속도 테스트

- 객체 크기, 스레드 수를 다르게 하여 new & delete 와 TLS 메모리풀의 속도를 비교함(사용 CPU : Intel Core i5-4460 3.20GHz)

스레드 1개가 10,000,000 번 할당하고 해제하는데 소요된 시간
(스레드 수: 2)



스레드 1개가 10,000,000 번 할당하고 해제하는데 소요된 시간
(스레드 수: 4)



(1) 객체 크기가 작을 경우, TLS 메모리풀은 new & delete보다 4.5배 빠름

(2) 객체 크기가 클 경우, TLS 메모리풀은 new & delete보다 8.5배 빠름

(3) 스레드 수가 2개일 때와 4개일 때 속도에 큰 차이가 없는데, 이유는 new & delete와 TLS 메모리풀 모두 유저모드 동기화객체를 사용하여 WaitOnAddress 방식으로 동기화를 수행하기 때문에 스레드 수에 대한 영향이 적기 때문이다.

7. A* 알고리즘 길찾기 프로그램

개발환경

- C++, Win32 API

개발기간

- 2022.07 ~ 2022.07

개요

- 2D 그리드 상에서 목적지로부터 도착점까지 A* 알고리즘으로 길을 찾아가는 프로그램 개발.
- 길찾기가 올바르게 작동하는지 눈으로 확인하기 위해 Win32 프로그램을 개발하여 검증함.
- 랜덤 지형을 생성하여 6시간 이상의 테스트 수행.

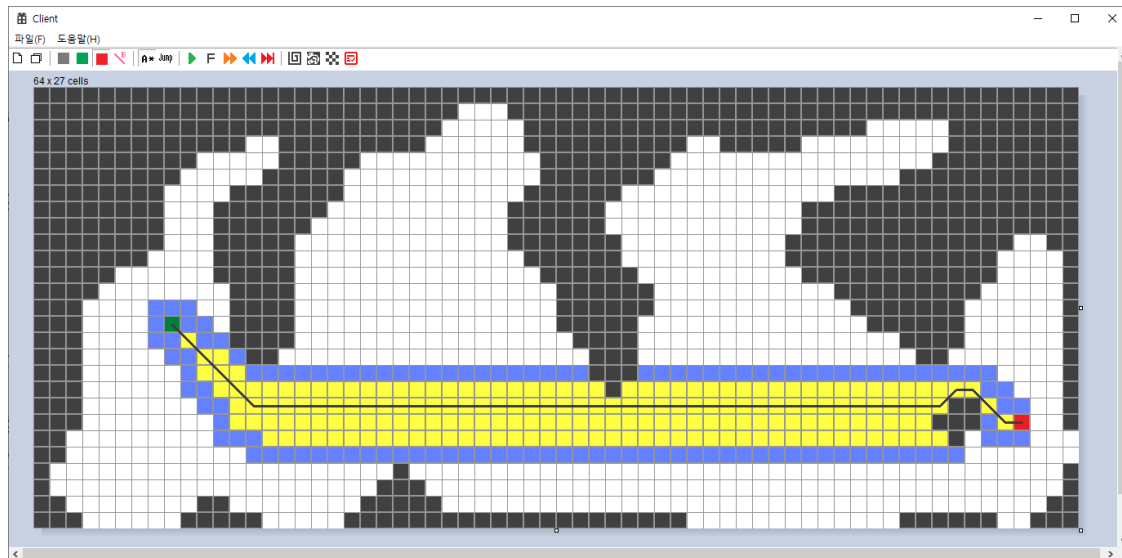
Win32 프로그램 모습

프로그램 시연영상:

https://youtu.be/5M_13YVESpw

A* 알고리즘 테스트 영상:

<https://youtu.be/opeyslOlnjQ>



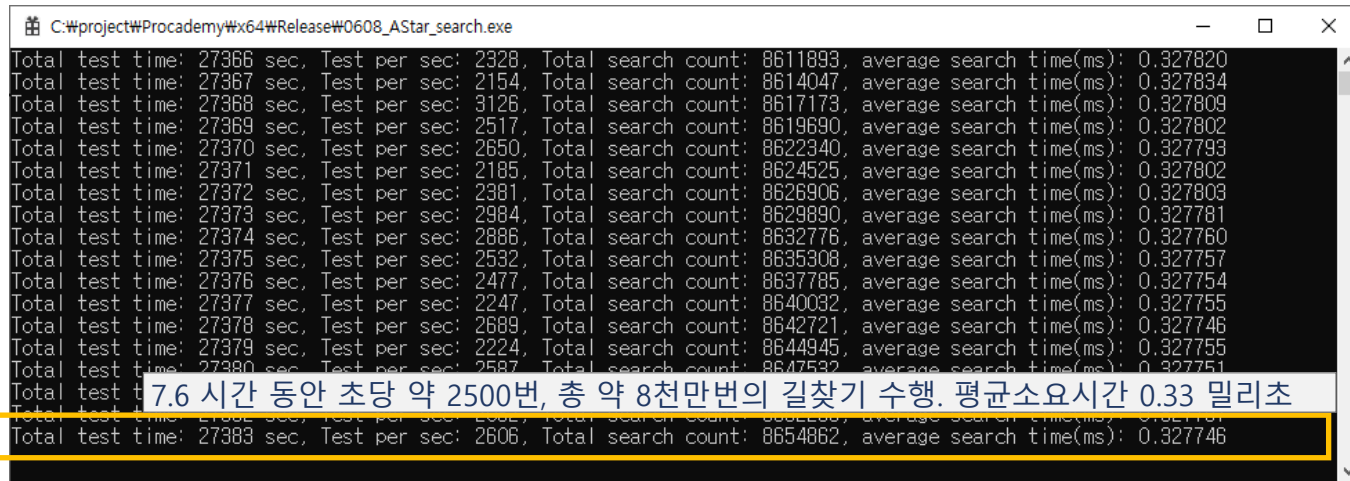
7. A* 알고리즘 길찾기 프로그램

알고리즘 테스트 조건

- 랜덤 생성된 지형 상에서 길찾기를 반드시 성공해야함.

테스트 결과

- 약 7시간 동안 8천만번의 길찾기를 수행하였고, 모두 성공하였음.



```
C:\project\Procademy\#x64\Release\#0608_AStar_search.exe
Total test time: 27366 sec, Test per sec: 2328, Total search count: 8611893, average search time(ms): 0.327820
Total test time: 27367 sec, Test per sec: 2154, Total search count: 8614047, average search time(ms): 0.327834
Total test time: 27368 sec, Test per sec: 3126, Total search count: 8617173, average search time(ms): 0.327809
Total test time: 27369 sec, Test per sec: 2517, Total search count: 8619690, average search time(ms): 0.327802
Total test time: 27370 sec, Test per sec: 2650, Total search count: 8622340, average search time(ms): 0.327793
Total test time: 27371 sec, Test per sec: 2185, Total search count: 8624525, average search time(ms): 0.327802
Total test time: 27372 sec, Test per sec: 2381, Total search count: 8626906, average search time(ms): 0.327803
Total test time: 27373 sec, Test per sec: 2984, Total search count: 8629890, average search time(ms): 0.327781
Total test time: 27374 sec, Test per sec: 2886, Total search count: 8632776, average search time(ms): 0.327760
Total test time: 27375 sec, Test per sec: 2532, Total search count: 8635308, average search time(ms): 0.327757
Total test time: 27376 sec, Test per sec: 2477, Total search count: 8637785, average search time(ms): 0.327754
Total test time: 27377 sec, Test per sec: 2247, Total search count: 8640032, average search time(ms): 0.327755
Total test time: 27378 sec, Test per sec: 2689, Total search count: 8642721, average search time(ms): 0.327746
Total test time: 27379 sec, Test per sec: 2224, Total search count: 8644945, average search time(ms): 0.327755
Total test time: 27380 sec, Test per sec: 2587, Total search count: 8647532, average search time(ms): 0.327751
Total test time: 27381 sec, Test per sec: 2506, Total search count: 8650119, average search time(ms): 0.327747
Total test time: 27382 sec, Test per sec: 2506, Total search count: 8652706, average search time(ms): 0.327746
Total test time: 27383 sec, Test per sec: 2606, Total search count: 8654862, average search time(ms): 0.327746
```

8. Jump Point Search 알고리즘 길찾기 프로그램

개발환경

- C++, Win32 API

개발기간

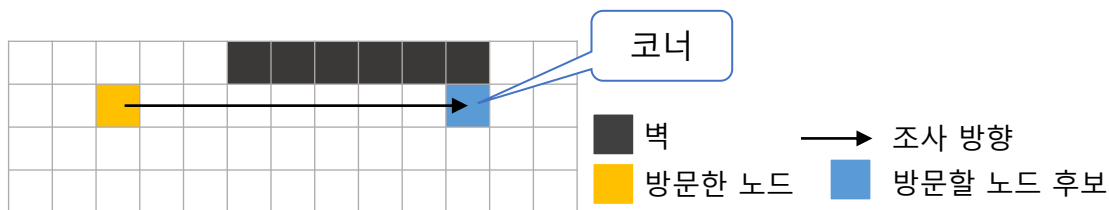
- 2022.07 ~ 2022.07

개요

- 2D 그리드 상에서 목적지로부터 도착점까지 Jump Point Search 알고리즘으로 길을 찾아가는 프로그램 개발.
- 길찾기가 올바르게 작동하는지 눈으로 확인하기 위해 Win32 프로그램을 개발하여 검증함.
- 랜덤 지형을 생성하여 6시간 이상의 테스트 수행.

Jump Point Search 알고리즘의 특징

- A* Search는 방문한 노드의 모든 인접노드가 "방문할 노드 후보"가 되어 방문해야될 노드의 수가 매우 많아진다(속도가느림).
- Jump Point Search는 이 단점을 보완하여 방문한 노드의 모든 직선/대각선 방향을 미리 조사하면서, **코너**를 마주칠 때만 "방문할 노드 후보"를 생성하여 방문해야될 노드 수를 획기적으로 줄였다(속도가빨라짐).



8. Jump Point Search 알고리즘 길찾기 프로그램

Win32 프로그램 개발

- 길찾기 결과에 오류가 없는지 눈으로 확인하기 위해 개발함.

알고리즘 테스트 조건

- 랜덤 생성된 지형 상에서 길찾기를 반드시 성공해야함.

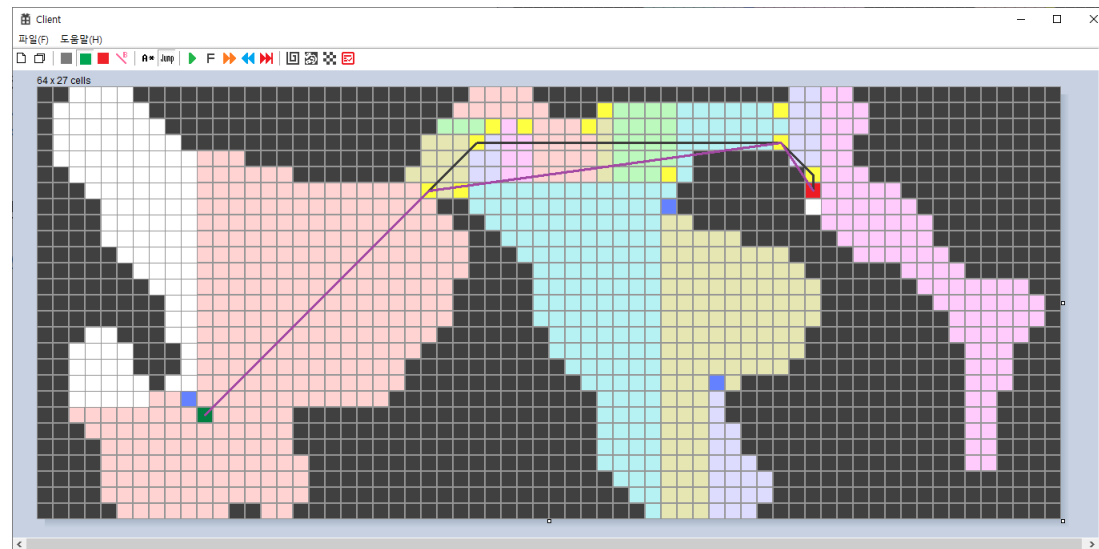
테스트 결과

- 약 8시간 동안 2억번의 길찾기를 수행하였고, 모두 성공하였음.

```
C:\project\Procademy\#x64\Release\#0608_AStar_search.exe
Total test time: 29888 sec, Test per sec: 11579, Total search count: 215022536, average search time(ms): 0.045478
Total test time: 29889 sec, Test per sec: 11624, Total search count: 215034160, average search time(ms): 0.045478
Total test time: 29890 sec, Test per sec: 11863, Total search count: 215046023, average search time(ms): 0.045478
Total test time: 29891 sec, Test per sec: 11613, Total search count: 215057636, average search time(ms): 0.045478
Total test time: 29892 sec, Test per sec: 11791, Total search count: 215069427, average search time(ms): 0.045478
Total test time: 29893 sec, Test per sec: 11040, Total search count: 215080467, average search time(ms): 0.045478
Total test time: 29894 sec, Test per sec: 11899, Total search count: 215092366, average search time(ms): 0.045478
Total test time: 29895 sec, Test per sec: 10853, Total search count: 215103219, average search time(ms): 0.045479
Total test time: 29896 sec, Test per sec: 12058, Total search count: 215115277, average search time(ms): 0.045479
Total test time: 29897 sec, Test per sec: 10536, Total search count: 215125813, average search time(ms): 0.045479
Total test time: 29898 sec, Test per sec: 10823, Total search count: 215136636, average search time(ms): 0.045479
Total test time: 29899 sec, Test per sec: 10420, Total search count: 215147056, average search time(ms): 0.045479
Total test time: 29900 sec, Test per sec: 10191, Total search count: 215179296, average search time(ms): 0.045479
```

8.3 시간 동안 초당 약 10000번, 총 약 2억번의 길찾기 수행. 평균소요시간 0.045 밀리초

Win32 프로그램 모습



프로그램 시연영상: <https://youtu.be/eEIFASsFCVM>

Jump Point Search 알고리즘 테스트 영상: <https://youtu.be/sA4Ts29EJpQ>

A* 알고리즘과의 속도 비교

알고리즘	평균속도(ms)
A* Search	0.327746
Jump Point Search	0.045479

- Jump Point Search가 A* Search에 비해 7.2배 더 빠른것을 확인하였음